

Package ‘TSOpt’

September 30, 2026

Type Package

Title Training Set Optimisation for Genomic Prediction in Any Crop

Version 0.5.0

Description Chooses which candidates to phenotype so that genomic prediction works best. One engine, built on rank-one (Sherman-Morrison) updates of the prediction error variance, covers inbred lines, doubled haploids, hybrids (parent x cross designs), clonal and polyploid crops, genebanks and sparse multi-environment testing, with a compiled engine and a low-rank engine for panels of tens of thousands of lines. Implements 17 criteria (CDmean, PEVmean, CDpop, worst-case CD, r-score, A-, D-, E- and G-optimality, expected genetic gain, coverage and more) with exact fixed effects, costs, budgets, quotas and historical data; protects major genes and haplotypes (a mixed-model scan, locus-weighted kernels and allele-carrier constraints); multi-environment sparse testing with environment-specific genetic covariances and residuals; multi-trait selection-index designs, also across environments and robust to uncertain genetic correlations, with the value of every trait and of every plot; genetically aware field layouts; bootstrap uncertainty of the plan; economic optimum of the training-set size; choice of whom to genotype; transfer between breeding programmes; designs robust to the model of gene action; a BrAPI connector to breeding databases; multi-objective Pareto fronts (NSGA-II); Bayesian designs over uncertain heritability; sample-size operating curves; expected top-k identification; two-stage adaptive designs by the knowledge gradient; optimum contribution selection of parents; and a long-term recurrent-selection simulator. Reads genotypes in VCF, HapMap, PLINK, DArT and numeric formats, diagnoses whether optimisation can help, compares plans against random and stratified sampling, and draws publication-ready figures. Free of charge for everyone; each user registers with PlantNura for a personal licence key.

License file LICENSE

Copyright PlantNura Consulting & Solutions. Free of charge for everyone under the TSOpt Free Licence; a personal licence key from PlantNura is required (tso_licence_request()). Third-party components: inst/THIRD-PARTY-NOTICES.

Encoding UTF-8

Depends R (>= 4.1.0)

Imports data.table,
ggplot2 (>= 3.5.0),

scales,
stats,
utils,
grDevices,
tools,
parallel

Suggests cluster,
patchwork,
ggrepel,
ragg,
svglite,
readxl,
rmarkdown,
knitr,
testthat (>= 3.0.0),
R.utils,
xfun,
jsonlite,
curl,
openssl,
writexl

VignetteBuilder knitr

Config/testthat/edition 3

Config/roxygen2/version 8.1.0

Roxygen list(markdown = TRUE)

Contents

tso_analysis_script	4
tso_backtest	4
tso_brapi_connect	6
tso_brapi_genome	7
tso_brapi_pheno	8
tso_brapi_studies	9
tso_brapi_trait	10
tso_brapi_variantsets	11
tso_brapi_write_list	11
tso_brapi_write_units	12
tso_cli	13
tso_colors	13
tso_compare	14
tso_contribution	15
tso_criteria	17
tso_crosses	17
tso_data	19
tso_design	23
tso_design_large	27
tso_diagnose	28
tso_docs	30
tso_economics	30

tso_evaluate	32
tso_explain	33
tso_export	33
tso_export_field	34
tso_genotype_set	35
tso_haplotypes	37
tso_imputation_plan	38
tso_kernels	40
tso_kinship	42
tso_layout	43
tso_licence	45
tso_licence_request	46
tso_major_effects	46
tso_major_scan	48
tso_manifest	50
tso_next_batch	50
tso_omics_kernel	52
tso_omics_plan	54
tso_pareto	55
tso_pareto_pick	58
tso_plan_json	59
tso_plot	59
tso_qc	60
tso_read	61
tso_report	63
tso_sample_size	64
tso_save_figure	65
tso_score	66
tso_season	67
tso_simulate	69
tso_simulate_cycles	70
tso_simulate_omics	71
tso_simulate_panel	72
tso_simulate_trait	73
tso_theme	74
tso_topk	75
tso_trait_value	76
tso_transfer	77
tso_uncertainty	79
tso_update	80
tso_variance	81
tso_verify	82
tso_write	83

tso_analysis_script	<i>The analysis model that matches the design</i>
---------------------	---

Description

Writes an R script that fits the model the plan was designed for, with the same fixed effects (intercept, environment or tester means, extra fixed effects) and the same relationship structure, in sommer, ASReml-R or rrBLUP. Fill in the phenotype file and run it once the trial is harvested.

Usage

```
tso_analysis_script(
  plan,
  file = NULL,
  engine = c("sommer", "asreml", "rrBLUP"),
  trait = "y"
)
```

Arguments

plan	A tso_plan.
file	Output .R file; NULL returns the code.
engine	"sommer", "asreml" or "rrBLUP".
trait	Name of the trait column in the phenotype file.

Value

The code, invisibly.

tso_backtest	<i>Forward validation: how each method would have done in your programme</i>
--------------	--

Description

Replays a programme's history, cycle by cycle, and scores training-set methods on what matters: the accuracy of predictions made only from data available at the time.

Usage

```
tso_backtest(
  geno,
  pheno,
  n,
  methods = c("random", "default"),
  traits = "y",
  scheme = c("within_cycle", "from_history"),
  min_history = 1,
  window = Inf,
```

```

    h2 = NULL,
    n_random = 5,
    seed = 1,
    verbose = TRUE
  )

```

Arguments

geno	Genotypes of the lines (tso_geno, a file, or a tso_data).
pheno	A data frame with one row per line: id, cycle (year, cohort; numeric or sortable) and the trait column(s).
n	Lines to choose per cycle: a number, or a fraction (< 1) of the candidates.
methods	Methods to compare: a character vector or a named list. Each element is "random", "default" (tso_design() defaults), a criterion name from tso_criteria(), a list of tso_design() arguments (e.g. list(criterion = "cdmean", h2 = 0.3)), or a function f(data, n, candidates, target, history) returning line IDs.
traits	Trait column(s) in pheno (default "y").
scheme	"within_cycle" or "from_history".
min_history	Cycles of history needed before a cycle is tested (default 1).
window	Use only the last window cycles as history (default all).
h2	Heritability for GBLUP; default REML on each training set.
n_random	Repeats of random methods.
seed	Random seed.
verbose	Print progress.

Details

Two schemes:

- "within_cycle" (default; the decision TSOpt supports): in cycle k, all lines phenotyped in earlier cycles are history; each method chooses n of cycle k's candidates to phenotype; GBLUP trained on history + chosen predicts the rest of cycle k, whose observed phenotypes are the truth.
- "from_history": each method chooses n lines from the earlier cycles as the training set (e.g. which historical lines to keep, re-genotype or re-test); GBLUP trained on them predicts all of cycle k.

GBLUP has a fixed effect per cycle (year effects), and the heritability is estimated by REML on each training set unless h2 is given. Accuracy is the predictive ability: the correlation of predictions with phenotypes within the test cycle (year effects cancel). Random methods are repeated n_random times and averaged.

Value

A tso_backtest object: results (one row per trait, test cycle and method: predictive ability, rank correlation, top-10% overlap, size of the training and test sets), summary (per method, with the paired difference from random and its 95% confidence interval across cycles).

Examples

```
hist <- tso_simulate_cycles(cycles = 4, n_candidates = 80, markers = 400, seed = 1)
bt <- tso_backtest(hist$geno, hist$pheno, n = 0.25, n_random = 2, verbose = FALSE)
bt
```

tso_brapi_connect	<i>Connect to a BrAPI v2 server</i>
-------------------	-------------------------------------

Description

Creates a small connection object for a server that speaks the Breeding API (BrAPI) version 2, such as BMS, Breedbase, GIGWA, DeltaBreed or the public test server. The server's /serverinfo is read to learn which endpoints it offers; servers without /serverinfo are tolerated.

Usage

```
tso_brapi_connect(
  url,
  token = NULL,
  page_size = 1000,
  timeout = 60,
  retries = 3,
  verbose = TRUE
)
```

Arguments

url	Base URL of the server. /brapi/v2 is added when missing, so "https://test-server.brapi.org" and "https://test-server.brapi.org/brapi/v2" are the same.
token	Optional access token. It is sent as the header Authorization: Bearer <token>.
page_size	Records requested per page (BrAPI pageSize).
timeout	Seconds to wait for one request, and for an asynchronous search to finish.
retries	How many times a failed request (network error, HTTP 429 or 5xx) is retried, with a growing pause between tries.
verbose	Print what was found.

Value

An object of class tso_brapi: a list with url, token, page_size, timeout, retries, calls (the services listed by /serverinfo, or NULL) and server (server name, if given).

See Also

[tso_brapi_geno\(\)](#), [tso_brapi_pheno\(\)](#), [tso_brapi_trait\(\)](#), [tso_brapi_studies\(\)](#), [tso_brapi_variantsets\(\)](#)

Examples

```
## Not run:
con <- tso_brapi_connect("https://test-server.brapi.org")
con
tso_brapi_studies(con)

## End(Not run)
```

tso_brapi_geno	<i>Read genotypes from a BrAPI v2 server</i>
----------------	--

Description

Downloads the genotype calls of one variant set and returns them as a `tso_geno` object (lines x markers, alternate-allele dosages), ready for `tso_data()`, `tso_kinship()` or `tso_design()`.

Usage

```
tso_brapi_geno(
  con,
  variant_set = NULL,
  germplasm = NULL,
  ...,
  method = c("auto", "allelematrix", "calls"),
  names = c("germplasm", "germplasmDbId", "callset"),
  ploidy = NULL,
  verbose = TRUE
)
```

Arguments

<code>con</code>	A connection from <code>tso_brapi_connect()</code> .
<code>variant_set</code>	The <code>variantSetDbId</code> to read. When <code>NULL</code> and the server has exactly one variant set, that one is used.
<code>germplasm</code>	Optional <code>germplasm</code> names or ids (or call set names or ids) to keep; all lines are kept when <code>NULL</code> .
<code>...</code>	Further query parameters passed to the genotype endpoint, e.g. <code>expandHomozygotes = TRUE</code> .
<code>method</code>	"auto", "allelematrix" or "calls".
<code>names</code>	Row names: "germplasm" (germplasm name, the default), "germplasmDbId" or "callset" (call set name). Use the same choice as <code>id</code> in <code>tso_brapi_pheno()</code> so that genotypes and phenotypes match.
<code>ploidy</code>	Ploidy, if you want to fix it instead of reading it from the genotype strings.
<code>verbose</code>	Print a short reading log.

Details

Two BrAPI routes are supported. `/allelematrix` (BrAPI 2.1) returns blocks of genotype strings (the GT data matrix), paged over call sets and variants; `/variantsets/<id>/calls` (BrAPI 2.0 and 2.1) returns one record per call. With `method = "auto"` the allele matrix is used when the server offers it, otherwise the calls endpoint. Genotype strings such as `"0/1"`, `"1|1"` (phased) or `"0/0/1/1"` are turned into counts of non-reference alleles; the ploidy is the number of alleles per call; `"./."` or `"."` are missing. Marker names, chromosomes and positions come from `/variants` (BrAPI start is 0-based, so `pos = start + 1`). Rows are named by germplasm: each call set's sample is looked up in `/samples` and its germplasm in `/germplasm`; when that fails the call set name is used.

Value

A `tso_geno` object (format "BrAPI"); its map has marker, chrom, pos, counted (ALT) and other (REF) when the server provides them, plus `variantDbId`; `callsets` holds the call set to germplasm table.

See Also

[tso_brapi_variantsets\(\)](#), [tso_read\(\)](#)

Examples

```
## Not run:
con <- tso_brapi_connect("https://test-server.brapi.org")
tso_brapi_variantsets(con)
geno <- tso_brapi_geno(con, variant_set = "variantset1")
geno

## End(Not run)
```

tso_brapi_pheno

Read phenotypes from a BrAPI v2 server

Description

Downloads observations from `/observations` and returns them as a long table, one row per observation. Studies are filtered on the server (`studyDbId`) and again locally; traits are matched locally against the observation variable name or id, so either can be given. If a server ignores or mishandles the study filter and returns nothing, all observations are read once and filtered locally (a message says so).

Usage

```
tso_brapi_pheno(
  con,
  study = NULL,
  trait = NULL,
  ...,
  germplasm = NULL,
  id = c("germplasm", "germplasmDbId"),
  verbose = TRUE
)
```

Arguments

con	A connection from tso_brapi_connect() .
study	Optional studyDbId(s) (or study names).
trait	Optional observation variable name(s) or id(s).
...	Further query parameters for /observations, e.g. observationLevel = "plot" or seasonDbId = "2023".
germplasm	Optional germplasm names or ids to keep.
id	What identifies a line in column id (and in the means): "germplasm" (germplasm name, the default) or "germplasmDbId". A warning is given when one name covers several germplasm ids, because their observations would then be pooled.
verbose	Print a short summary.

Details

When exactly one trait is present in the result, the attribute "y" holds the germplasm means (see [tso_brapi_trait\(\)](#)).

Value

A data.frame with columns id (germplasm name, or id when the name is missing), trait, value (numeric; values that are not numbers become NA), study, germplasmDbId, traitDbId, observationUnitDbId, season and raw (the value as sent). With one trait, attribute "y" is the named vector of germplasm means.

See Also

[tso_brapi_trait\(\)](#), [tso_brapi_studies\(\)](#)

Examples

```
## Not run:
con <- tso_brapi_connect("https://test-server.brapi.org")
ph <- tso_brapi_pheno(con, study = "study1")
head(ph)

## End(Not run)
```

tso_brapi_studies	<i>List the studies on a BrAPI server</i>
-------------------	---

Description

List the studies on a BrAPI server

Usage

```
tso_brapi_studies(con, ...)
```

Arguments

con	A connection from <code>tso_brapi_connect()</code> .
...	Query parameters for /studies, e.g. commonCropName = "Rice", programDbId = "p1" or trialDbId = "t1".

Value

A data.frame with studyDbId, studyName, trialDbId, trialName, locationName, seasons, commonCropName, startDate and active.

Examples

```
## Not run:
con <- tso_brapi_connect("https://test-server.brapi.org")
tso_brapi_studies(con)

## End(Not run)
```

tso_brapi_trait	<i>One trait from a BrAPI server as a named vector</i>
-----------------	--

Description

Reads one trait with `tso_brapi_pheno()` and averages the observations of each germplasm (replicates, plots, and all requested studies are pooled with a simple mean; missing and non-numeric values are dropped). The result is a named numeric vector ready for `tso_evaluate()` or `tso_major_scan()`. For designs with strong study effects, compute adjusted means (BLUES) from the long table instead.

Usage

```
tso_brapi_trait(con, study = NULL, trait = NULL, ...)
```

Arguments

con	A connection from <code>tso_brapi_connect()</code> .
study	Optional studyDbId(s).
trait	Observation variable name or id. May be NULL when the selection contains only one trait.
...	Passed to <code>tso_brapi_pheno()</code> .

Value

A named numeric vector of germplasm means; attribute "n" gives the number of observations averaged for each germplasm.

Examples

```
## Not run:
con <- tso_brapi_connect("https://test-server.brapi.org")
y <- tso_brapi_trait(con, study = "study1", trait = "Corn Stalk Height")

## End(Not run)
```

`tso_brapi_variantsets` *List the variant sets (genotyping data sets) on a BrAPI server*

Description

List the variant sets (genotyping data sets) on a BrAPI server

Usage

```
tso_brapi_variantsets(con, ...)
```

Arguments

<code>con</code>	A connection from <code>tso_brapi_connect()</code> .
<code>...</code>	Query parameters for <code>/variantsets</code> , e.g. <code>studyDbId = "s1"</code> .

Value

A data.frame with `variantSetDbId`, `variantSetName`, `studyDbId`, `referenceSetDbId`, `callSetCount` and `variantCount`.

Examples

```
## Not run:
con <- tso_brapi_connect("https://test-server.brapi.org")
tso_brapi_variantsets(con)

## End(Not run)
```

`tso_brapi_write_list` *Write a plan to a BrAPI database: a germplasm list*

Description

Saves the lines of a plan (or any IDs) as a germplasm list on a BrAPI v2 server (POST `/lists`), so the field team finds the training set in BMS, Breedbase or any BrAPI database. With `resolve = TRUE`, germplasm names are first matched to database IDs (POST `/search/germplasm`).

Usage

```
tso_brapi_write_list(
  con,
  x,
  name,
  description = NULL,
  resolve = TRUE,
  dry_run = FALSE
)
```

Arguments

con	A connection from tso_brapi_connect() .
x	A tso_plan, a tso_layout, or a character vector of germplasm names or IDs.
name	List name.
description	Optional list description.
resolve	Match germplasm names to germplasmDbIds first (default TRUE); FALSE sends the IDs as they are.
dry_run	Return the request body without sending it.

Value

The created list (its listDbId and the server's reply), or the request body with dry_run = TRUE.

See Also

[tso_brapi_write_units\(\)](#) for the plots of a study.

tso_brapi_write_units *Write the plots of a plan or layout to a BrAPI study*

Description

Creates one observation unit per plot in an existing study (POST /observationunits): germplasm, plot number, block, replicate, and row and column when a field layout is given; checks are marked as such. Phenotypes recorded on these units come back to TSOpt with [tso_brapi_trait\(\)](#).

Usage

```
tso_brapi_write_units(
  con,
  x,
  study,
  prefix = "plot",
  resolve = TRUE,
  dry_run = FALSE
)
```

Arguments

con	A connection from tso_brapi_connect() .
x	A tso_layout (with field positions) or a tso_plan.
study	The studyDbId of the study (trial at one location).
prefix	Prefix of the observation-unit names.
resolve	Match germplasm names to germplasmDbIds first (default TRUE); FALSE sends the IDs as they are.
dry_run	Return the request body without sending it.

Value

The server's reply (or the body with dry_run = TRUE).

tso_cli

*Run TSOpt from the command line***Description**

The engine behind the tsopt command-line tool (inst/cli/tsopt), the Docker image and the Python client: parses arguments, runs the analysis, writes the result. See tsopt help.

Usage

```
tso_cli(args = commandArgs(TRUE))
```

Arguments

args Character vector of arguments (as from commandArgs(TRUE)).

Details

Commands: design, diagnose, backtest, variance, crosses, help. Options take the form --name value; lists are comma separated; ID lists can be given as a file with one ID per line.

Value

The result object, invisibly. Output files are written as asked (--out: .json for a plan file, .csv for the table).

Examples

```
tso_cli("help")
```

tso_colors

*Colours used throughout TSOpt***Description**

One colour per role, used in every figure, so a reader learns the code once: training units are purple, targets teal, other candidates light grey, historical data amber, the random baseline mid grey and the optimised plan orange. The group palette is Okabe-Ito (colour-blind safe).

Usage

```
tso_colors(which = c("roles", "groups"), n = 8)
```

Arguments

which "roles" (default) or "groups".
n For "groups": how many colours.

Value

A named character vector of hex colours.

Examples

```
tso_colors()
tso_colors("groups", 4)
```

tso_compare	<i>Compare a plan with the alternatives a breeder would otherwise use</i>
-------------	---

Description

Puts the optimised plan side by side with random sampling (a distribution, not a single draw), stratified random sampling, the "most related to the target" heuristic (average kinship; known to be harmful), partitioning around medoids, and any sets you supply, all scored on the same target.

Usage

```
tso_compare(
  plan,
  methods = c("random", "stratified", "kinship", "pam"),
  others = NULL,
  n_random = 100,
  seed = 1
)
```

Arguments

plan	A tso_plan from tso_design() .
methods	Any of "random", "stratified", "kinship", "pam".
others	A named list of extra training sets (unit IDs), e.g. the list you would have planted anyway.
n_random	Number of random (and stratified) draws.
seed	Random seed.

Value

A tso_comparison object (a data frame with one row per method).

Examples

```
panel <- tso_simulate_panel(n = 150, markers = 500, seed = 1)
plan <- tso_design(panel, n = 30, verbose = FALSE)
tso_compare(plan)
```

tso_contribution	<i>Optimum contribution selection: which parents, and how much of each</i>
------------------	--

Description

tso_contribution() chooses the parents of the next generation and their genetic contributions so that the expected genetic value of the offspring is as high as possible while the loss of genetic diversity is held at an agreed rate (Meuwissen 1997). It is the natural next step after genomic prediction: breeding values from TSOpt, the same relationship matrix.

Usage

```
tso_contribution(
  ebv,
  K,
  max_coancestry = NULL,
  delta_F = NULL,
  n_max = NULL,
  min_contribution = 0,
  sex = NULL,
  n_matings = NULL
)
```

Arguments

ebv	Named numeric vector of breeding values (names = IDs), or a data frame with a column id and a column ebv, gebv or predicted (e.g. tso_evaluate(...)\$predictions).
K	Relationship matrix with row and column names covering the IDs of ebv (e.g. data\$K_lines).
max_coancestry	Upper limit C for the group coancestry $c'Kc/2$.
delta_F	Alternatively, the accepted rate of inbreeding per generation (see Details).
n_max	Optional maximum number of parents (heuristic, see Details).
min_contribution	Optional minimum contribution of a selected parent (heuristic, see Details). Default 0.
sex	Optional named vector or factor (names = IDs) with values "M" / "F"; each sex then contributes one half.
n_matings	Optional number of matings. Contributions are then rounded to integer numbers of matings per parent (largest remainder): parent i takes part in about $2Nc_i$ of the N matings, and with sex each sex totals N .

Details

Problem. With breeding values b , relationships K and contributions c ,

$$\max_c c'b \quad \text{s.t.} \quad c'Kc/2 \leq C, \quad c \geq 0, \quad 1'c = 1$$

(with sex: the contributions of each sex sum to 1/2 instead). The group coancestry of the parents, $c'Kc/2$, is the expected inbreeding of their offspring under random mating.

Coancestry limit. Give `max_coancestry = C` directly, or a rate of inbreeding `delta_F`, in which case

$$C_t = C_{t-1} + \Delta F (1 - C_{t-1}), \quad C_{t-1} = \bar{K}/2,$$

where $\bar{K}/2 = \text{mean}(K) / 2$ is the mean coancestry of the current candidates (the group coancestry of contributing all of them equally). This is the definition of the rate of inbreeding, $\Delta F = (F_t - F_{t-1}) / (1 - F_{t-1})$, applied to coancestry, which is the inbreeding of the next generation (Meuwissen 1997; Grundy et al. 1998). With a genomic relationship matrix centred on the current candidates (as from `tso_kinship()`), $\bar{K} \approx 0$, so $C = \Delta F$: diversity is measured relative to the current population. If neither is given, `delta_F = 0.01` (an effective population size of 50) is used.

Solver (exact). For a price λ of coancestry, the problem $\max c'b - \lambda c'Kc/2$ subject to the equality constraints and $c \geq 0$ is a convex quadratic programme, solved exactly by a primal active-set method: the equality-constrained KKT system is solved on the free set, units that would turn negative are fixed at zero, and units at zero with a positive marginal value (negative bound multiplier) are released, until the KKT conditions hold. λ is then found by bisection so that the coancestry constraint binds, and the last step interpolates between the two bracketing solutions (both optimal for the limiting λ) so that $c'Kc/2 = C$ to machine precision. If putting everything on the best candidate(s) already satisfies the limit, that is the answer ($\lambda = 0$); if even the minimum-coancestry contributions exceed it, those are returned with a warning.

`n_max` (at most that many parents) and `min_contribution` are combinatorial and handled heuristically: the largest contributions are kept (or those below the minimum dropped) and the problem is re-solved exactly on the remaining support.

Feeding it from TSOpt. Use breeding values from a GBLUP fit, e.g. `ev <- tso_evaluate(plan, y)` then `ebv <- setNames(ev$predictions$predicted, ev$predictions$id)` (or pass `ev$predictions` directly), and `K = data$K_lines` from the same `tso_data()` object.

Value

A `tso_contribution` object: `contributions` (data frame `id`, `ebv`, `contribution`, `n_matings`, `selected`, `sex`), `mean_ebv` ($c'b$), `coancestry` ($c'Kc/2$), `limit` (C), `current_coancestry` (C_{t-1}), `delta_F` (expected rate of inbreeding), `lambda`, `kkt` (optimality residuals), `truncation` (the same number of top-ranked parents with equal contributions) and `spec`.

References

Meuwissen T.H.E. (1997) *Genet Sel Evol* 29: 225-231. Grundy B., Villanueva B., Woolliams J.A. (1998) *Genet Res* 72: 159-168.

Examples

```
panel <- tso_simulate_panel(n = 120, markers = 400, seed = 1)
d <- tso_data(panel, verbose = FALSE)
tr <- tso_simulate_trait(panel, h2 = 0.5, seed = 2)
ebv <- setNames(tr$g, tr$id) # stand-in for GBLUP breeding values
oc <- tso_contribution(ebv, d$K_lines, delta_F = 0.02)
oc

plot(oc)
```

tso_criteria

*The criteria TSOpt knows, and which one to use***Description**

TSOpt picks for you: `tso_design()` uses **robust CDmean**: CDmean with the intercept as an exact fixed effect, averaged over $h^2 = 0.2, 0.5$ and 0.8 , found by the greedy rank-one search. It had the best realised accuracy in the head-to-head benchmark (benchmark/ in the repository). The Rank-One Tabu Search (search = "tabu") reaches higher criterion values but, like any hard optimiser of one assumed model, did not predict better. The rest are there for specialist goals and for comparison. This table says what each one does and when it is worth switching.

Usage

```
tso_criteria(tier = NULL)
```

Arguments

tier Optionally, only one tier: "Recommended", "Specialist", "Classical", "Model-free" or "Baseline".

Value

A data frame (printed as a table).

Examples

```
tso_criteria()
```

tso_crosses

*Predict crosses: progeny mean, progeny variance and usefulness***Description**

Ranks candidate crosses among inbred parents by the usefulness criterion $U = \mu + i\sigma$: the expected mean of the selected fraction of their progeny. The progeny variance comes from the marker effects and the genetic map (see Details), so two crosses with the same mean can differ widely in what they can deliver.

Usage

```
tso_crosses(
  geno,
  parents = NULL,
  crosses = NULL,
  effects = NULL,
  pheno = NULL,
  cM_per_Mb = NULL,
  progeny = c("DH", "RIL"),
  select = 0.1,
  accuracy = 1
)
```

Arguments

geno	Genotypes of the candidate parents (homozygous lines) and, when effects are estimated, of the phenotyped lines.
parents	IDs of the candidate parents (default: all lines).
crosses	Optional two-column matrix or data frame of crosses to evaluate (default: every pair of parents).
effects	Marker effects (named numeric vector, per copy of the counted allele), or NULL to estimate them from pheno.
pheno	Named numeric vector of phenotypes (names = line IDs), used when effects is NULL.
cM_per_Mb	Map density when the map has no cM column.
progeny	"DH" (doubled haploids from the F1, default) or "RIL".
select	Fraction of progeny selected (default 0.1), which sets the selection intensity i .
accuracy	Accuracy of selection among the progeny (default 1: on true genetic values); $U = \mu + i \cdot accuracy \cdot \sigma$.

Details

Marker effects are given (effects, e.g. from BayesB or `tso_major_effects()`), or estimated from pheno by ridge regression (rrBLUP, REML heritability). Ridge regression shrinks effects, so the predicted variances are too small in absolute terms; their ranking is what matters for choosing crosses.

Genetic map. Positions in centiMorgan are taken from a cM column of the map, or from base-pair positions times cM_per_Mb (for example about 0.5 in maize, 0.2 in wheat, 4 in rice; there is no safe default). Recombination follows Haldane's map function.

Designing for cross prediction. Accurate cross prediction needs accurate GEBVs and marker effects for the candidate parents: design the training set with `tso_design(..., target = parents)`.

Value

A `tso_crosses` object: table (one row per cross: parents, mean, sd, usefulness, rank), effects and settings.

References

Schnell FW (1983) Probleme der Elternwahl. Ein Ueberblick. Arbeitstagung der Saatzuchtleiter, Gumpenstein. Lehrermeier C, Teyssedre S, Schoen CC (2017) Genetics 207:1651-1661.

Examples

```
panel <- tso_simulate_panel(n = 120, markers = 400, seed = 1)
tr <- tso_simulate_trait(panel, h2 = 0.6, seed = 1)
cr <- tso_crosses(panel, parents = rownames(panel$dosage)[1:10], pheno = setNames(tr$y, tr$id))
cr
```

tso_data

Assemble everything the design engine needs

Description

tso_data() is the front door of TSOpt. Give it your genotypes in any format (or a relationship matrix, or a pedigree), say which breeding system you work in, and it builds the units that can be phenotyped and the relationship matrix between them.

Usage

```
tso_data(
  geno = NULL,
  kinship = NULL,
  pedigree = NULL,
  system = c("inbred", "clonal", "outcross", "genebank", "hybrid", "testcross"),
  groups = NULL,
  families = NULL,
  cost = NULL,
  ploidy = NULL,
  qc = TRUE,
  base_freq = NULL,
  dominance = NULL,
  dom_share = 0.2,
  pools = NULL,
  sca_share = 0.2,
  crossable = NULL,
  cross_groups = NULL,
  testers = NULL,
  envs = NULL,
  env_cor = 0.6,
  env_noise = NULL,
  traits = NULL,
  trait_h2 = NULL,
  gcor = NULL,
  trait_cost = NULL,
  plot_cost = 0,
  kernels = NULL,
  kernel_share = NULL,
  haplotypes = NULL,
  variance = NULL,
  implicit = NULL,
  major = NULL,
  major_share = NULL,
  major_model = c("random", "fixed"),
  min_carriers = 2,
  verbose = TRUE
)
```

Arguments

geno	Genotypes: a file path (any format <code>tso_read()</code> accepts), a <code>tso_geno</code> object, or a matrix.
kinship	Optional relationship matrix to use instead of (or without) genotypes.
pedigree	Optional pedigree (id, parent 1, parent 2) used when no genotypes or kinship are given, and by the pedigree and single-step kernels (kernels =).
system	One of "inbred", "clonal", "outcross", "genebank", "hybrid".
groups	Optional group labels per line (e.g. subpopulations, programmes), named by line ID.
families	Optional family labels per line, named by line ID.
cost	Optional cost per unit (named by unit ID, or in unit order). Defaults to 1 for every unit.
ploidy	Ploidy, if it cannot be detected from the data.
qc	Run <code>tso_qc()</code> with its defaults (TRUE) or pass a list of arguments for it, or FALSE to skip.
base_freq	Base-population allele frequencies (see <code>tso_qc()</code>).
dominance	Add a dominance kernel (TRUE by default for diploid clonal crops and for hybrids' SCA).
dom_share	Share of genetic variance given to dominance when dominance = TRUE (default 0.2).
pools	For system = "hybrid": a list of two ID vectors (female pool, male pool), or of three or more heterotic groups (named); crosses are then made between the groups in cross_groups.
sca_share	For hybrids: share of hybrid genetic variance due to SCA (default 0.2; GCA of the two pools share the rest equally).
crossable	For hybrids: parents that may be used in training crosses (default: all). Hybrids involving other parents can be targets but not training units.
cross_groups	For three or more heterotic groups: the pairs of groups that are crossed, as a list of length-2 vectors of group names (default: every pair). The genetic value of the single cross $i \times j$ is $a_i + a_j + s_{ij}$ with covariance $\sigma_a(K_{ik} + K_{il} + K_{jk} + K_{jl}) + \sigma_s(K_{ik}K_{jl} + K_{il}K_{jk})$.
testers	For system = "testcross": the tester IDs. Testers that are genotyped get their genomic relationships (for the SCA part); otherwise they are taken as unrelated.
envs	Optional environment names, for line x environment designs.
env_cor	Genetic correlation between environments: a single value, a correlation matrix, or a genetic covariance matrix (environment-specific genetic variances on its diagonal), e.g. estimated from earlier trials.
env_noise	Relative residual variance of each environment (named by environment; default 1 everywhere). A noisy site (2) is worth half a plot at a clean one (1): heteroscedastic multi-environment designs.
traits	Optional trait names, for multi-trait (line x trait) designs: a unit is then one trait measured on one line, and <code>tso_design()</code> can optimise a selection index with index =.
trait_h2	Entry-mean heritability of each trait (same order as traits).
gcor	Genetic correlation matrix between traits (default: identity).

trait_cost	Cost of measuring each trait on one line (default 1).
plot_cost	Cost of growing a line at all, paid once per line the first time any trait is measured on it (default 0).
kernels	Genetic kernels to combine: kernel names ("A", "D", "AA", "AD", "DD", "gaussian", "exponential", "A_ped", "D_ped", "AA_ped", "H"; see tso_kernels()), a tso_kernels() object, a tso_omics_kernel() object (transcriptome, metabolome, spectra; the genomic kernel is added, and genotyped lines without a profile are predicted from genomics), or a named list of your own matrices (name the additive one "A"), or a list mixing all of these (<code>list("A", "AA", NIRS = om)</code>). The relationship matrix becomes $K = \sum_k w_k K_k$. Replaces kinship and dominance.
kernel_share	Share of genetic variance of each kernel (named, or in the order of kernels), or "robust" (default with several kernels): the design is then averaged over non-additive shares of 10%, 30% and 50% (split equally over the non-additive kernels), and the plan reports the 30% centre. See <i>Genetic kernels</i> .
haplotypes	Haplotype blocks for the haplotype-coverage criterion, the per-block coverage report of every plan and the "HAP" kernel: TRUE (LD blocks), a list of tso_haplotypes() arguments (e.g. <code>list(method = "fixed", window = 3)</code>), or a tso_haplotypes object.
variance	A tso_variance() fit of historical data. It supplies what would otherwise be assumed: the heritabilities and genetic correlations of traits, the genetic covariance and residual variances of envs, or the shares of kernels; and, for robust designs, scenarios drawn from the estimates' sampling distribution (for the heritability, its 10%, 50% and 90% quantiles replace $h^2 = 0.2, 0.5, 0.8$).
implicit	Keep the unit covariance of traits and environments in factorised form ($\Omega_E \otimes G_T \otimes K$, never formed as one matrix). NULL (default) switches it on above 8,000 line x trait x environment cells; designs are identical to the dense form, and 1,000 lines x 15 environments fit in memory. Not for hybrids yet.
major	Major-effect loci to protect in the design: marker names, a tso_major_scan() result (from phenotypes), a tso_major_effects() result or a table of marker effects you already have (from a GWAS, QTL study or earlier model: no phenotypes needed), a tso_haplotypes() object, or <code>list(markers = , haplotypes =)</code> . See <i>Major genes and haplotypes</i> .
major_share	Share of genetic variance at the major loci together (for <code>major_model = "random"</code>). Default: the scan's or the marker effects' estimate, or 0.3.
major_model	"random" (default): the relationship matrix becomes $(1 - s)K_{poly} + sK_{major}$, so the design values sampling every allele of the major loci. "fixed": the loci are fixed effects in the model (Bernardo 2014); tso_design() then optimises the polygenic CDmean with the exact projection for them.
min_carriers	Minimum number of training units carrying each allele (haplotype) of the major loci that the targets carry (default 2; 0 turns the constraint off). Can be changed per design in tso_design() .
verbose	Print what was done.

Details

Breeding systems:

- "inbred" (default): lines, doubled haploids, RILs. A unit is a line.

- "clonal": clones (potato, cassava, sugarcane, banana). A unit is a clone; the kernel adds a dominance part for diploids.
- "outcross": families or individuals of outcrossing species.
- "genebank": accessions of a diversity panel; the design defaults to an untargeted reference panel with stratification.
- "testcross": candidate lines evaluated as testcrosses with one or more testers (the other heterotic group). A unit is line $\times$ tester, with genetic value $GCA(\text{line}) + SCA(\text{line} \times \text{tester})$ and covariance $(v_{GCA}J + v_{SCA}K_{\text{testers}}) \otimes K_{\text{lines}}$; testers act like environments (a tester mean each, sparse allocation with `max_per_line`), and `tso_design()` predicts the lines' GCA by default.
- "hybrid": single crosses between two pools. A unit is a hybrid female $\times$ male; its genetic value is $GCA + GCA + SCA$, built from the parents' relationships.

Any system can be combined with `envs` for sparse multi-environment testing: a unit is then a line in one environment. `traits` and `envs` combine: a unit is then one trait measured on one line in one environment, with genetic covariance $\Omega_E \otimes G_T \otimes K$, a mean per trait and environment (exact fixed effects), and the plot cost paid once per line per environment.

Value

A `tso_data` object.

Genetic kernels

A kernel is a covariance model for genetic values. The additive kernel alone assumes all genetic variance is additive. With `kernels =` the total genetic value is modelled as a sum of independent parts (additive, dominance, epistatic, or non-parametric), each with its share of the genetic variance. `tso_design()` then predicts either the total genetic value (`value = "total"`, the worth of a variety or clone) or the breeding value (`value = "breeding"`, the additive part, what a parent passes on). Epistatic values are shared mainly by close relatives, so designs for them favour training sets close to the targets. Which parts matter depends on the crop: inbred lines carry no dominance, so `A` and `AA` are the relevant kernels; clones inherit everything; hybrids already model dominance and epistasis between pools through `SCA`.

Major genes and haplotypes

A genome-wide kernel gives every marker the same weight, so a major gene is a vanishing part of it and `CDmean` does not care whether the training set contains both of its alleles. When one allele is missing its effect cannot be estimated, and every target carrying it is predicted from the polygenic background alone. With `major =`, `TSOpt` (1) gives the loci the variance they explain (or treats them as fixed effects) and (2) requires every allele class carried by the targets to appear at least `min_carriers` times in the training set. Every plan also reports the genome-wide allele and haplotype coverage of the targets.

The loci can come from your phenotypes (`tso_major_scan()`) or from marker effects you already have (`tso_major_effects()`); either way each locus is weighted in the major-locus kernel by the share of variance it explains.

Examples

```
panel <- tso_simulate_panel(n = 120, markers = 500, seed = 1)
tp <- tso_data(panel)
tp
```

tso_design

*Design a training set: which units to phenotype***Description**

The one function most users need. Give it your data and how many units you can phenotype (or a budget); get back a plan saying which ones, how good the choice is compared with random sampling, and what to watch out for.

Usage

```
tso_design(
  data,
  n = NULL,
  budget = NULL,
  target = NULL,
  goal = c("accuracy", "families", "information", "gain", "correlation",
    "reference_panel"),
  criterion = NULL,
  select_top = 0.1,
  h2 = NULL,
  candidates = NULL,
  must_include = NULL,
  exclude = NULL,
  history = NULL,
  history_weight = 1,
  stratify = c("auto", "groups", "none"),
  group_quota = NULL,
  max_per_family = NULL,
  max_per_parent = NULL,
  max_per_env = NULL,
  reps = 1,
  index = NULL,
  predict = c("all", "H2", "H1", "H0"),
  search = c("auto", "greedy", "greedy+exchange", "tabu", "ga", "swap"),
  fixed = c("intercept", "none"),
  iterations = 2000,
  max_pass = NULL,
  max_contrasts = Inf,
  min_carriers = NULL,
  env_target = "mean",
  max_per_line = NULL,
  min_per_env = NULL,
  restarts = 0,
  cores = 1,
  direction = c("maximize", "minimize"),
  trait_uncertainty = c("auto", "none"),
  kernels = NULL,
  value = c("total", "breeding", "gca"),
  n_random = 100,
  seed = 1,
```

```

    verbose = TRUE
  )

```

Arguments

data	A tso_data object, or anything <code>tso_data()</code> accepts (genotype file, tso_geno object, marker matrix).
n	Number of units to phenotype.
budget	Alternatively, a budget in the same currency as the unit costs given to <code>tso_data()</code> . Units are then chosen by gain per unit cost.
target	Units you want to predict (IDs). NULL means untargeted: the whole pool.
goal	What you are trying to do; see the table above.
criterion	Expert override of the criterion (see above).
select_top	For goal = "gain": the fraction you will select (default 0.1).
h2	Assumed entry-mean heritability. The default (NULL) is the <i>robust CDmean</i> design: the criterion is averaged over $h2 = 0.2, 0.5$ and 0.8 , which won the head-to-head benchmark on six public panels (see benchmark/). It falls back to $h2 = 0.5$ when an expert search, a criterion other than CDmean / PEVmean / CDpop, or several traits are used. Give one value to design for a known heritability, or several values for your own prior grid.
candidates	Units that may be phenotyped. Default: every unit that is not a target (for hybrids, only crosses between crossable parents).
must_include	Units that must be in the set (checks, parents, lines already planted). They count towards n and budget.
exclude	Units that cannot be phenotyped (no seed, lost, off-type).
history	Units already phenotyped in earlier seasons. They inform the design at no cost and are not counted in n.
history_weight	Weight of historical records relative to new ones (1 = as informative as a new plot; smaller for older or less related seasons).
stratify	"auto" (default): stratify when the panel has discrete groups; "groups": always stratify by the groups in data; "none".
group_quota	Optional named vector of the number of units per group.
max_per_family, max_per_parent, max_per_env	Optional caps.
reps	Maximum number of plots per unit (replicates). With $reps > 1$, the engine may put several plots on the same unit when that is more informative.
index	For multi-trait data (see traits in <code>tso_data()</code>): the selection-index weights, named by trait, per genetic standard deviation. The design then maximises the reliability of the index (index CDmean), or the expected gain on the index with goal = "gain".
predict	For hybrids: which untested hybrids matter most: "all", "H2" (both parents tested), "H1" (one new parent), "H0" (both parents new).
search	"greedy+exchange" (default), "greedy", or "swap" (random swaps; used automatically for criteria without a fast update).

fixed	Fixed effects of the prediction model. "intercept" (default) uses the exact fixed-intercept form of CDmean / PEVmean / CDpop (Rincent's "exact" CD-mean); "none" treats the data as having a known mean. For any other fixed effects give the names of columns of data\$units (factors become indicator columns, e.g. "group"), or a numeric matrix / data frame with one row per unit (or per line, matched by row names): trial, check or covariate effects. Environment means (multi-environment data) and fixed major loci (tso_data(major_model = "fixed")) are added automatically. Every fixed effect is handled exactly by a flat prior inside the rank-one engine (the contrasts stay on the genetic values), at no extra cost per step.
iterations	Iterations for swap search.
max_pass	Maximum number of exchange passes. Default: 3 for pools up to 1,500 units, 1 above (exchange rarely adds more than 0.2% there).
max_contrasts	Optional cap on the number of target contrasts used during the search (a random subset; reported values always use all targets). Only for very large pools: subsetting biases the search towards the sampled targets.
min_carriers	Major loci (see major in tso_data()): the minimum number of training units carrying each allele class that the targets carry. Default: the value given to tso_data() (2). 0 switches the constraint off.
env_target	Multi-environment data: what the design should predict well. "mean" (default): the lines' genetic values averaged over the environments (performance across the target population of environments); "cells": every line x environment cell; or the name of one environment (e.g. a new site with its correlation in env_cor).
max_per_line	Multi-environment (or multi-trait) data: the maximum number of environments (traits) in which one line is tested. 1 gives a fully sparse allocation.
min_per_env	Multi-environment data: the minimum number of units in every environment (a single number or one per environment). Default (NULL): 2 per environment. Environment means are fixed effects, so the first plot in an environment only estimates its mean and a one-step search would never open a new environment; two plots each remove that trap, and the search then allocates the rest freely. Use 0 to switch this off.
restarts	Extra multi-start runs of the exchange / tabu search from perturbed starts (default 0). The best set by the criterion is kept.
cores	Cores for restarts (forked processes; 1 on Windows).
direction	For a user-supplied criterion function: "maximize" (default) or "minimize". TSOpt warns when a function looks like it is being optimised in the wrong direction.
trait_uncertainty	Multi-trait data: "auto" (default) hedges the design over uncertain trait parameters. The index criterion is averaged over five scenarios: the given heritabilities and genetic correlations, heritabilities 40% lower and higher, and correlations halved towards 0 or moved halfway towards +/-1. Correlations estimated from small trials are noisy; this is the multi-trait counterpart of robust CDmean. "none" designs for the given values only.
kernels	Optional kernels to hedge over, from "additive", "dominance" and "gaussian" (the RKHS kernel, which captures epistasis). The criterion is averaged over the kernels (and over the heritability grid), so the set does not over-fit one model of gene action. Line-level data with genotypes only.

value	What the design predicts. For testcross data (system = "testcross") the default is "gca", the lines' general combining ability (the part of testcross performance a line passes to every hybrid); "total" predicts the testcross values themselves. Otherwise, when the genetic model has non-additive kernels (tso_data(kernels =)): "total" (default), the total genetic value (the worth of a variety, clone or line itself), or "breeding", the breeding value (the additive part, what a parent passes on: use it to choose parents). The data are modelled with every kernel either way; only the targets change. With kernel_share = "robust" in <code>tso_data()</code> the design averages over the share scenarios.
n_random	Number of random training sets drawn as a baseline.
seed	Random seed for the baseline and swap search.
verbose	Print progress messages.

Details

Goals (breeder language) and the criterion each one uses:

goal	criterion	use when
"accuracy" (default)	CDmean (Rincent et al. 2012)	general use
"families"	CDpop: contrasts between family means (Rincent et al. 2017)	targets are crosses,
"information"	D-optimality	no target set is known
"gain"	expected mean genetic value of the top select_top fraction (experimental)	truncation selection
"correlation"	r-score (Ou & Liao 2019)	direct expected correlation
"reference_panel"	untargeted CDmean, stratified when the panel is structured	reusable panels, genetic

Experts can set criterion directly ("cdmean", "pevmean", "cdpop", "dopt", "gain", "rscore") or pass their own function `f(selected_ids, data)` returning a number to maximise.

How it works. The engine keeps the posterior covariance of all genetic values, $A(S) = (K^{-1} + D_S/\lambda)^{-1}$, and updates it with one rank-one (Sherman-Morrison) step each time a unit is added or removed, so the relationship matrix is never inverted. Greedy forward selection builds the set; an exchange pass (modified Fedorov) then tries to improve it.

Value

A `tso_plan` object. Print it for a summary, plot it with `tso_plot()`, export it with `tso_export()`. `plan$selected` lists the units with their expected reliability and their **value**: the exact CDmean lost if that plot is dropped (value, and value_pct as a share of the design's CDmean), so plots can be cut or swapped knowingly when seed or land runs short (`tso_plot(plan, "value")`).

References

Rincent R. et al. (2012) Genetics 192: 715-728. Akdemir D., Sanchez J.I., Jannink J.-L. (2015) Genet Sel Evol 47: 38. Ou J.-H., Liao C.-T. (2019) Theor Appl Genet 132: 3151-3161.

Examples

```
panel <- tso_simulate_panel(n = 200, markers = 600, seed = 1)
plan <- tso_design(panel, n = 40)
plan
head(plan$selected)
```

tso_design_large	<i>Design a training set in a very large panel (no $N \times N$ kinship)</i>
------------------	---

Description

`tso_design()` works on the full relationship matrix, which is out of reach beyond ~15,000 lines (20,000 lines = 3.2 GB for K alone). `tso_design_large()` gives the same robust CDmean design for panels of 20,000-100,000+ lines by working in marker (feature) space: it never forms an $N \times N$ matrix.

Usage

```
tso_design_large(
  geno,
  n,
  target = NULL,
  h2 = NULL,
  rank = 300,
  candidates = NULL,
  must_include = NULL,
  exclude = NULL,
  max_contrasts = Inf,
  n_random = 20,
  sketch = 5000,
  seed = 1,
  verbose = TRUE
)
```

Arguments

geno	A <code>tso_geno</code> object, a dosage matrix (lines x markers), or anything <code>tso_read()</code> accepts. <code>tso_qc()</code> is run (silently) when the object has no stored base allele frequencies.
n	Number of lines to phenotype (including <code>must_include</code>).
target	Lines you want to predict (IDs or indices). NULL means untargeted: the whole pool.
h2	Assumed entry-mean heritability. NULL (default) is robust CDmean, averaged over $h2 = 0.2, 0.5$ and 0.8 , as in <code>tso_design()</code> . Give one value, or several for your own grid.
rank	Number of features (components of K) kept.
candidates	Lines that may be phenotyped. Default: every line that is not a target.
must_include	Lines that must be in the set; they count towards n.
exclude	Lines that cannot be phenotyped.
max_contrasts	Optional cap on the number of target contrasts used during the search (a random subset). The search cost does not depend on the number of contrasts here, so the default uses all of them; reported values always use all targets.
n_random	Number of random training sets drawn as a baseline.
sketch	Lines used to find the marker subspace when the panel is larger (default 5,000).
seed	Random seed (SVD sketch, contrast subset and baseline).
verbose	Print progress messages.

Details

How it works. The VanRaden kinship is $K = ZZ'/c$ with $Z = X - kp$. A rank-rank feature matrix F with $K \approx FF'$ is taken from a randomised truncated SVD of Z (subspace iteration with one power step, Halko et al. 2011), or $F = Z$ exactly when there are no more markers (or lines) than rank. K is scaled to mean diagonal 1, as in `tso_kinship()`, and modelled as $K \approx FF' + \text{diag}(e)$: what the features miss is kept as line-specific variance e_i (it adds to the noise of a phenotyped line and to the variance of a target), so the reported CDmean stays close to the exact one even when the features capture only part of $\text{trace}(K)$. Genetic values are $g = F\beta + \delta$, $\beta \sim N(0, I)$, and the engine keeps the rank x rank posterior covariance of β , updated by one rank-one (Sherman-Morrison) step per added line. The fixed intercept gets a flat prior (an extra all-ones feature), exactly as the dense engine uses $K + c11'$. With an exact F the selection is identical to `tso_design(..., search = "greedy")`.

Cost. Building F takes a few passes over the dosage matrix; with more than sketch lines, the marker subspace is found on a random subset of sketch lines and every line is then projected on it (one pass over all lines). One greedy step costs $O(Nr)$ per heritability value, independent of the number of target contrasts. F takes $8Nr$ bytes (100,000 lines x 300 = 240 MB); peak memory is about three times that, plus the dosage matrix. There is no exchange pass (in large pools it rarely adds more than 0.2%).

The share of $\text{trace}(K)$ captured by the features is reported; if it is low ($< 70\%$), increase rank. The line-specific part of a phenotyped line is treated as noise (its own observed residual is not credited), so the reported CDmean is slightly conservative (about 1% at 80% captured).

Value

A `tso_plan_large` object with selected (order, id), value (CDmean at the median h^2 , by h^2 , the random baseline, percentile and expected accuracy gain), coverage, spec (including `variance_captured`, the share of $\text{trace}(K)$ captured by the features), trace (criterion along the greedy search), verdict and time.

References

Rincent R. et al. (2012) *Genetics* 192: 715-728. Halko N., Martinsson P.G., Tropp J.A. (2011) *SIAM Review* 53: 217-288.

Examples

```
panel <- tso_simulate_panel(n = 600, markers = 800, seed = 1)
plan <- tso_design_large(panel, n = 60, rank = 100, verbose = FALSE)
plan
head(plan$selected)
```

tso_diagnose

Should I optimise, and how many units do I need?

Description

`tso_diagnose()` answers the questions to ask *before* designing:

Usage

```
tso_diagnose(
  data,
  n = NULL,
  target = NULL,
  h2 = 0.5,
  sizes = NULL,
  n_random = 50,
  seed = 1
)
```

Arguments

data	A tso_data object or anything <code>tso_data()</code> accepts.
n	The number of units you plan to phenotype (used for the verdict). Default: 20% of the candidates.
target	Optional target units (IDs).
h2	Assumed entry-mean heritability.
sizes	Training-set sizes for the size curve.
n_random	Random sets drawn at each size.
seed	Random seed.

Details

1. **Is the panel optimisable?** It compares a greedy CDmean set with random sets of the same size. If the gain is small, random or stratified sampling is the honest recommendation.
2. **What is the structure?** Principal components and clustering of the relationship matrix, with a verdict: continuum, mild structure or discrete groups.
3. **How many units?** A size curve of expected accuracy for random and optimised sets, with the effective number of chromosome segments (M_e) fitted to the curve rather than taken from a formula.
4. **Anything odd?** Near-duplicate lines and the effective number of independent lines.

Value

A tso_diagnosis object. Print it, or plot it with `tso_plot()`.

Examples

```
panel <- tso_simulate_panel(n = 200, markers = 500, seed = 1)
dg <- tso_diagnose(panel, n = 40)
dg
```

tso_docs	<i>Open the TSOpt guides, tutorial and manual</i>
----------	---

Description

The PDFs come with the package (release builds). `tso_docs()` lists them; `tso_docs("tutorial")` opens one. "walkthrough-rmd" gives the path of the walkthrough's R Markdown file, to render on your own data.

Usage

```
tso_docs(which = NULL)
```

Arguments

which	NULL (list), or one of "tutorial", "manual", "walkthrough", "guide1" to "guide5", "walkthrough-rmd".
-------	--

Value

The file path(s), invisibly.

Examples

```
tso_docs()
```

tso_economics	<i>How many plots pay off? The economic size of a training set</i>
---------------	--

Description

`tso_economics()` turns the accuracy curve of `tso_sample_size()` into money: for every training-set size it weighs the value of the genetic gain that the predictions deliver against the cost of the plots, and reports the size that maximises the net value, for an optimised design and for random sampling.

Usage

```
tso_economics(
  data,
  cost_per_plot,
  value_per_sd,
  select_top = 0.1,
  sizes = NULL,
  fixed_cost = 0,
  target = NULL,
  h2 = NULL,
  sigma_g = 1,
  horizon = 1,
  n_random = 20,
  seed = 1
)
```

Arguments

data	A tso_data object or anything <code>tso_data()</code> accepts.
cost_per_plot	Cost of phenotyping one unit. If missing (or NULL), the mean of <code>data\$units\$cost</code> (the costs given to <code>tso_data()</code>).
value_per_sd	Value of one unit of genetic gain in the selected fraction (one genetic SD when $\sigma_g = 1$), per cycle.
select_top	Fraction of the targets selected on the predictions (default 0.1).
sizes	Optional training-set sizes at which random sampling is evaluated (passed to <code>tso_sample_size()</code>); the optimised curve is exact at every size up to <code>max(sizes)</code> . Default: about 30 sizes spread over the whole candidate pool.
fixed_cost	Cost paid once whatever the size (trial set-up, genotyping already budgeted, analysis). Shifts the net value, never the optimum.
target	Optional target units (IDs), as in <code>tso_sample_size()</code> .
h2	Assumed heritability; NULL averages over 0.2, 0.5 and 0.8.
sigma_g	Genetic standard deviation of the trait, in the units that <code>value_per_sd</code> is priced in (default 1: value per genetic SD).
horizon	Number of selection cycles (or years) over which the training set is used (default 1).
n_random	Random training sets per evaluated size.
seed	Random seed.

Details

Model. With accuracy $r(n) = \sqrt{CDmean(n)}$ (the exact greedy curve of `tso_sample_size()` for the optimised design; the mean of random sets, interpolated linearly between the evaluated sizes, for random sampling), selecting the best fraction $select_top = p$ of the targets on their predictions gives the response per cycle

$$R(n) = i(p) r(n) \sigma_g, \quad i(p) = \phi(\Phi^{-1}(1 - p))/p.$$

The value of the design is $horizon \times value_per_sd \times R(n)$, its cost $fixed_cost + cost_per_plot \times n$, and the net value the difference. `value_per_sd` is what one unit of genetic gain in the selected fraction is worth (one genetic SD when $\sigma_g = 1$), e.g. the extra revenue per hectare times the hectares sown with the selected material.

The optimum is the size with the largest net value on the curve; there the value of the last plot, $V(n) - V(n - 1)$, is at least its cost and the value of one more plot is at most its cost. The break-even size is the smallest size whose value covers the costs.

Value

A `tso_economics` object: `curve` (data frame with, for every size n , `acc_opt`, `acc_random`, `value_opt`, `value_random`, `cost`, `net_opt`, `net_random`, `marginal_opt`, `marginal_random`), `optimum` (list with, for `opt` and `random`: `n`, `net`, `value`, `cost`, `accuracy`, `marginal_value`, `break_even`, and `advantage = net value of the optimised design at its optimum minus that of random sampling at its own`), `intensity`, `sample_size` (the underlying `tso_sample_size()` result) and `spec`.

Examples

```
panel <- tso_simulate_panel(n = 150, markers = 400, seed = 1)
ec <- tso_economics(panel, cost_per_plot = 40, value_per_sd = 20000, n_random = 5)
ec

plot(ec)
```

tso_evaluate

Evaluate a training set once phenotypes are in

Description

Fits GBLUP on the training units and reports how well it predicted the target units, with the metrics a breeder actually cares about: predictive ability (correlation), rank correlation, NDCG of the top fraction, and **realised gain**: how much better the selected top fraction is than the target mean.

Usage

```
tso_evaluate(
  x,
  y,
  train = NULL,
  target = NULL,
  truth = NULL,
  h2 = NULL,
  top = 0.1,
  forward = FALSE
)
```

Arguments

x	A tso_plan, or a tso_data object together with train.
y	Named numeric vector of phenotypes (names = unit IDs). Training units must be present; targets are evaluated where present.
train	Training units, when x is a tso_data object.
target	Units to evaluate on. Default: the plan's targets (or all units not in training).
truth	Optional named vector of true genetic values (simulations); when given, metrics are computed against it instead of y.
h2	Heritability for GBLUP. NULL (default) estimates it by REML from the training data (bounded to 0.02-0.95).
top	Fraction selected, for NDCG and realised gain (default 0.1).
forward	Is this forward prediction (later season)?

Details

Call it with forward = TRUE when the target was phenotyped in a later season or environment than the training set; within-season numbers are labelled optimistic, because forward prediction is typically about half as accurate.

Value

A tso_evaluation object.

Examples

```
panel <- tso_simulate_panel(n = 200, markers = 600, seed = 1)
tr <- tso_simulate_trait(panel, h2 = 0.5, seed = 2)
plan <- tso_design(panel, n = 50, verbose = FALSE)
tso_evaluate(plan, y = setNames(tr$y, tr$id), truth = setNames(tr$g, tr$id))
```

tso_explain

Explain exactly what a plan did

Description

Prints, in plain language, every choice behind a plan: the breeding system, the criterion and its formula, the heritability and the resulting λ , targets, candidates, constraints, stratification and search. The returned list is the complete specification, so the plan can be reproduced or edited.

Usage

```
tso_explain(plan, steps = TRUE)
```

Arguments

plan	A tso_plan.
steps	If TRUE (default), also print the algebra step by step.

Value

The specification (a list), invisibly.

Examples

```
panel <- tso_simulate_panel(n = 100, markers = 400, seed = 1)
plan <- tso_design(panel, n = 20, verbose = FALSE)
tso_explain(plan)
```

tso_export

Export a plan for the field team

Description

Writes the selected units (or every unit with its role) to CSV, ready for a field book or trial-management system.

Usage

```
tso_export(plan, file = NULL, what = c("selected", "all"))
```

Arguments

plan	A tso_plan.
file	Output file (.csv). If NULL, nothing is written.
what	"selected" (default) or "all" units with their role and expected reliability.

Value

The table, invisibly.

Examples

```
panel <- tso_simulate_panel(n = 100, markers = 400, seed = 1)
plan <- tso_design(panel, n = 20, verbose = FALSE)
tab <- tso_export(plan, file.path(tempdir(), "trial.csv"))
```

tso_export_field	<i>Export a plan or field layout for field tools</i>
------------------	--

Description

Writes the training set (a `tso_design()` plan) or its field layout (a `tso_layout()` object) in the format a field tool imports:

- "fieldbook": Field Book (PhenoApps) field file: plot_id, row, column, plot, block, rep, accession_name, entry_type.
- "fieldhub": FieldHub entry list: ENTRY, NAME (checks first), to generate augmented, p-rep or alpha-lattice designs there.
- "breedbase": Breedbase trial-upload sheet: plot_name, accession_name, plot_number, block_number, is_a_control, rep_number, row_number, col_number.
- "csv": the plain table (as `tso_export()`).

Usage

```
tso_export_field(
  x,
  file = NULL,
  format = c("fieldbook", "fieldhub", "breedbase", "csv"),
  trial = "TSopt"
)
```

Arguments

x	A tso_plan or tso_layout.
file	Output file (.csv). NULL returns the table only.
format	"fieldbook", "fieldhub", "breedbase" or "csv".
trial	Trial or study name, used in plot names and IDs.

Details

A plan without a layout has no field positions: fieldbook and breedbase then number the plots in the plan's order. Replicated lines (a partially replicated layout) get one row per plot.

Value

The table, invisibly.

Examples

```
panel <- tso_simulate_panel(n = 100, markers = 300, seed = 1)
plan <- tso_design(panel, n = 20, verbose = FALSE)
lay <- tso_layout(plan, rows = 5, cols = 5, checks = "CHK1", check_reps = 2, iterations = 2000)
head(tso_export_field(lay, format = "fieldbook", trial = "YT2027"))
tso_export_field(plan, format = "fieldhub")
```

tso_genotype_set

Whom to genotype: choose the individuals to genotype at high density

Description

tso_genotype_set() answers the question that comes *before* phenotyping: with money to genotype only *n* individuals at high density (an imputation reference panel, or simply a genotyping budget), which ones should they be? It works on the relationships known before genotyping: the pedigree (tso_data(pedigree =)), a cheap low-density kinship (tso_data(kinship =)) or low-density genotypes.

Usage

```
tso_genotype_set(
  data,
  n,
  candidates = NULL,
  must_include = NULL,
  method = c("coverage", "cdmean", "dopt"),
  target = NULL,
  n_random = 50,
  seed = 1
)
```

Arguments

data	A tso_data object (built with pedigree =, kinship = or low-density genotypes), anything <code>tso_data()</code> accepts, or a pedigree data frame (id, parent 1, parent 2).
n	Number of individuals to genotype (including must_include).
candidates	Individuals that can be genotyped (IDs; default: all, e.g. excluding animals or plants with no DNA left).
must_include	Individuals genotyped anyway (already genotyped, or required). They count towards n.

method	"coverage" (default), "cdmean" or "dopt".
target	Individuals whose genotypes you want to impute or predict (IDs; default: everyone).
n_random	Random sets of the same size used as a baseline.
seed	Random seed for the baseline.

Details

Relationships are used as correlations, $R = cov2cor(K)$ (for a pedigree without inbreeding this is the numerator relationship itself), so 0.5 is a parent or full sib and 0.25 a grandparent, half sib or uncle.

Methods.

- "coverage" (default): facility location. Maximises the mean, over the target individuals, of the relationship to the closest genotyped one (a genotyped individual covers itself with 1). The objective is monotone and submodular, so the greedy set is within $1 - 1/e$ of the best possible; it finds the key ancestors that tie the pedigree together.
- "cdmean": the package's CDmean greedy (centred contrasts over the targets, exact fixed intercept), treating a genotyped individual as an almost noise-free observation of its genetic value (relative noise 0.05): information spreads to relatives through the relationships.
- "dopt": D-optimality, $\log \det(I + K_{SS}/0.05)$: the most informative, least redundant set, without regard to any target.

Every set is compared with `n_random` random sets of the same size and with the usual "key ancestors by contribution" heuristic: the individuals with the largest row sums of the relationship matrix over the targets (the most connected ones).

Value

A `tso_genotype_set` object: selected (data frame: order, id, forced, gain = marginal gain in mean coverage when the individual was added, row_sum = its total relationship to the targets), coverage (for the targets not genotyped: best = relationship to the closest genotyped individual, mean, quantiles, share_025, share_050, objective = mean coverage over all targets), comparison (data frame: the set, the most-connected heuristic and the random mean, with mean coverage and shares), random_best (the pooled best-relative values of the random sets, for the figure), index and spec.

Examples

```
set.seed(1)
ped <- data.frame(id = paste0("F", 1:8), p1 = NA, p2 = NA)
for (g in 1:2) {
  par <- ped$id
  kid <- data.frame(id = paste0("G", g, "_", 1:12), p1 = sample(par, 12, TRUE),
                    p2 = sample(par, 12, TRUE))
  ped <- rbind(ped, kid)
}
d <- tso_data(pedigree = ped, verbose = FALSE)
gs <- tso_genotype_set(d, n = 6)
gs

plot(gs)
```

tso_haplotypes

Haplotype blocks and haplotype alleles

Description

Builds haplotype blocks (runs of adjacent markers) and the haplotype alleles of every line in them. The result protects major-gene haplotypes (`tso_data()` `major =`), defines the haplotype relationship matrix (`tso_kernels()` "HAP"), the haplotype-coverage criterion (`tso_design()` `criterion = "haplotype"`) and the per-block coverage report of every plan (`tso_data(haplotypes =)`).

Usage

```
tso_haplotypes(
  geno,
  method = c("fixed", "window", "ld"),
  around = NULL,
  blocks = NULL,
  window = 2,
  kb = 50,
  r2 = 0.3,
  max_markers = 20,
  min_freq = 0.02
)
```

Arguments

<code>geno</code>	Genotypes (<code>tso_geno</code> , a <code>tso_data</code> with genotypes, or anything <code>tso_read()</code> accepts).
<code>method</code>	"fixed" (default), "window" or "ld".
<code>around</code>	Marker names: keep only the blocks around them.
<code>blocks</code>	Your own blocks (named list of marker names) or a PLINK <code>.blocks</code> / <code>.blocks.det</code> file.
<code>window</code>	For "fixed": markers on each side of the centre (default 2, i.e. 5-marker blocks).
<code>kb</code>	For "window": block width in kilobases (default 50).
<code>r2, max_markers</code>	For "ld": the LD threshold (default 0.3) and the largest block (default 20 markers).
<code>min_freq</code>	Haplotypes rarer than this are pooled as "rare" in the relationship matrix and in major-gene classes (default 0.02). They are always kept for coverage.

Details

Blocks (method):

- "fixed": tiles of $2 * \text{window} + 1$ adjacent markers (default 5).
- "window": tiles of kb kilobases along each chromosome.
- "ld": adjacent markers joined while their mean r^2 with the block reaches r2 (default 0.3), up to `max_markers` markers: blocks follow the linkage disequilibrium of your panel, like the blocks of Haploview or `plink --blocks` (Gabriel et al. 2002), but by a simpler greedy rule.

- `blocks` =: your own blocks, as a named list of marker names, or the path of a PLINK `.blocks` or `.blocks.det` file.
- `around` =: only the blocks that contain these markers (for "fixed", window markers on each side of each).

Haplotypes. Inbred lines and doubled haploids are homozygous, so the multi-locus string is the haplotype (material with at least 99% homozygous calls counts as homozygous; the rare heterozygous calls are rounded). For heterozygous material, a phased VCF (0|1) gives one haplotype per chromosome copy (kept by `tso_read()` and `tso_qc()`). Without phase, the local multi-locus genotype stands in for the pair; phase the file first (Beagle 5, SHAPEIT, Eagle) for true haplotypes.

Value

A `tso_haplotypes` object: `blocks` (markers per block), `info` (chromosome, start, end, markers, haplotypes per block), `alleles` (lines x blocks labels), `hap` (for homozygous or phased data, one lines x blocks matrix of haplotype strings per chromosome copy), `freq` (haplotype frequencies per block), `kind` ("homozygous", "phased" or "unphased") and `method`.

References

Gabriel SB et al. (2002) Science 296:2225-2229.

Examples

```
panel <- tso_simulate_panel(n = 100, markers = 300, seed = 1)
hp <- tso_haplotypes(panel, method = "ld")
hp
tso_haplotypes(panel, around = c("m50", "m200"))
```

tso_imputation_plan *Plan genotyping, imputation and phenotyping under one budget*

Description

For each combination of reference-set size (`n_ref`, lines genotyped at high density) and low-density panel size (`ld_markers`), on genotypes you already have at high density:

1. `ld_markers` markers are chosen evenly along each chromosome (minor allele frequency at least 0.05);
2. `n_ref` reference lines are chosen by relationship coverage (`tso_genotype_set()`, or at random);
3. every other line is imputed from its `k` nearest reference lines, measured on the low-density markers of each window of window markers (weighted by similarity; for inbred lines this is close to LinkImpute's LD-kNNi, Money et al. 2015);
4. the budget left after genotyping buys $(\text{budget} - \text{cost}) / \text{plot_cost}$ plots (at most 90% of the lines, so some are left to predict); `tso_design()` chooses them on the imputed data;
5. the plan is scored by its **realised reliability**: GBLUP predicts the unphenotyped lines (or target) with the imputed relationship matrix, while the genetic values follow the true one. For centred target contrasts c ,

$$\frac{(c' \hat{K}_{TS} \hat{V}^{-1} K_{ST} c)^2}{c' \hat{K}_{TS} \hat{V}^{-1} (K_{SS} + \sigma^2 I) \hat{V}^{-1} \hat{K}_{ST} c' K_{TT} c},$$

the squared correlation of predicted and true contrasts, averaged. It equals CDmean when the imputed matrix is the true one, and falls with imputation error and with fewer plots.

Usage

```
tso_imputation_plan(
  geno,
  budget,
  cost_hd,
  cost_ld,
  plot_cost,
  n_ref = NULL,
  ld_markers = NULL,
  reference = c("coverage", "random"),
  target = NULL,
  h2 = 0.5,
  k = 10,
  window = 40,
  seed = 1
)
```

Arguments

geno	High-density genotypes of the candidates (tso_geno, a file, or a tso_data).
budget	Total budget, in the units of the costs.
cost_hd, cost_ld	Cost of one high- and one low-density genotype.
plot_cost	Cost of one phenotyping plot.
n_ref, ld_markers	Values to try (all combinations are evaluated).
reference	"coverage" (default) or "random" reference lines.
target	Lines to predict (default: the lines not phenotyped).
h2	Heritability for the design and its score.
k, window	Neighbours and window size (in markers) of the imputation.
seed	Random seed.

Details

The table also reports the imputation accuracy (correlation of imputed with true dosages over the imputed calls) and the fidelity of the relationship matrix (correlation of its off-diagonal entries with the true ones). A plan that genotypes everyone at high density is included for comparison.

Value

A tso_imputation_plan object: table (one row per plan: genotyping cost, plots, imputation accuracy, relationship fidelity, the CDmean the design expects on the imputed data, and the realised reliability), best.

References

Money D et al. (2015) G3 5:2383-2390.

Examples

```
panel <- tso_simulate_panel(n = 150, markers = 600, seed = 1)
ip <- tso_imputation_plan(panel, budget = 3000, cost_hd = 30, cost_ld = 10, plot_cost = 20,
                          n_ref = c(30, 60), ld_markers = c(60, 150))
ip
```

tso_kernels	<i>Genetic kernels: additive, dominance, epistasis, pedigree and single-step</i>
-------------	--

Description

Builds the relationship matrices (kernels) for the genetic models TSOpt can design for, each scaled to mean diagonal 1. Pass the result, or just the kernel names, to [tso_data\(\)](#) as `kernels =`, with the share of genetic variance each explains (`kernel_share`).

Usage

```
tso_kernels(
  geno = NULL,
  pedigree = NULL,
  which = NULL,
  bandwidth = 1,
  blend = 0.05,
  base_freq = NULL,
  haplotypes = NULL
)
```

Arguments

<code>geno</code>	Genotypes (<code>tso_geno</code> , a file, or a <code>tso_data</code> with genotypes).
<code>pedigree</code>	Pedigree: data frame with columns <code>id</code> , <code>parent 1</code> , <code>parent 2</code> (unknown parents NA or 0).
<code>which</code>	Kernels to build (see Details). Default: "A", plus "D" and "AA" for diploid marker data, or "A_ped" for a pedigree only.
<code>bandwidth</code>	Multiplier of the median distance for the Gaussian and exponential kernels (default 1).
<code>blend</code>	Weight of A22 in the blended G used by "H" (default 0.05).
<code>base_freq</code>	Optional base allele frequencies for "A" and "D".
<code>haplotypes</code>	For "HAP": a tso_haplotypes() object, or a list of its arguments (e.g. <code>list(method = "fixed", window = 3)</code>). Default: LD blocks.

Details

Marker kernels:

- "A": additive, VanRaden (2008), any ploidy.
- "D": dominance, Vitezica et al. (2013), diploids.

- "AA", "AD", "DD": pairwise epistasis as Hadamard (element-wise) products of A and D (Henderson 1985; Jiang & Reif 2015). Relationships are roughly squared, so they fall off faster with distance: epistatic values are shared mainly by close relatives.
- "gaussian", "exponential": non-parametric kernels on standardised markers, $\exp(-d^2/h)$ and $\exp(-d/h)$, with h the median squared (or plain) distance times bandwidth (Gianola & van Kaam 2008; Perez-Elizalde et al. 2015). They capture additive and epistatic similarity together.

Pedigree kernels (from pedigree, columns id, parent 1, parent 2):

- "A_ped": numerator relationship matrix.
- "D_ped": dominance relationship, $d_{ij} = (a_{s_i s_j} a_{d_i d_j} + a_{s_i d_j} a_{d_i s_j})/4$ (Cockerham 1954; the non-inbred approximation).
- "AA_ped": $A_{ped} \circ A_{ped}$.

Haplotypes:

- "HAP": the haplotype relationship matrix. Every haplotype block (see `tso_haplotypes()`) is treated as a multi-allelic locus whose alleles are its haplotypes; their dosages are centred and cross-multiplied as VanRaden does for SNPs. Haplotypes carry identity by descent that single SNPs miss when blocks are long. Blocks come from haplotypes (default: LD blocks).

Single-step:

- "H": the relationship matrix of Legarra et al. (2009) for a pedigree in which only some individuals are genotyped. Genotyped individuals get their genomic relationships, ungenotyped ones relationships propagated through the pedigree, so candidates that are not genotyped yet (new crosses) can be designed for. G is scaled to A22 (Vitezica et al. 2011) and blended with it (blend, default 5%; Aguilar et al. 2010).

Value

A `tso_kernels` object: K, a named list of matrices (mean diagonal 1), and info, a table of the kernels.

References

Henderson CR (1985) J Anim Sci 60:111-117. Jiang Y, Reif JC (2015) Genetics 201:759-768. Legarra A, Aguilar I, Misztal I (2009) J Dairy Sci 92:4656-4663. Vitezica ZG, Varona L, Legarra A (2013) Genetics 195:1223-1230. Gianola D, van Kaam JBCHM (2008) Genetics 178:2289-2303.

See Also

`tso_data()` (kernels, kernel_share), `tso_design()` (value).

Examples

```
panel <- tso_simulate_panel(n = 80, markers = 400, seed = 1)
kk <- tso_kernels(panel, which = c("A", "D", "AA", "gaussian"))
kk
tp <- tso_data(panel, kernels = kk, verbose = FALSE)
```

tso_kinship

*Relationship (kinship) matrices***Description**

Builds the relationship matrix that the design engine works on.

Usage

```
tso_kinship(
  geno,
  method = c("additive", "dominance", "pedigree"),
  base_freq = NULL,
  pedigree = NULL,
  scale = TRUE,
  ridge = 1e-04,
  weights = NULL
)
```

Arguments

geno	A tso_genotype object (after <code>tso_qc()</code>) or a dosage matrix.
method	"additive", "dominance" or "pedigree".
base_freq	Optional named vector of base allele frequencies. The vector stored by <code>tso_qc()</code> carries the counted allele of each marker (as the attribute "counted"), and genotypes are re-oriented to it, so frequencies stay about the same allele across cycles even for letter-coded files (HapMap, allele-pair tables), which are coded by each file's own minor allele.
pedigree	For method = "pedigree": a data frame with columns id, parent 1, parent 2 (unknown parents as NA or 0).
scale	If TRUE (default), rescale so the mean diagonal is 1. The heritability used by the design engine is then the entry-mean heritability, whatever the ploidy or inbreeding of the panel.
ridge	Small value added to the diagonal for numerical stability.
weights	Optional marker weights for the additive kernel, named by marker (unnamed: in column order): $G = ZWZ' / (k \sum wp(1 - p))$. Use it to up-weight markers near known QTL (e.g. $-\log_{10} p$ from <code>tso_major_scan()</code>), or give weight 0 to drop markers (e.g. major loci modelled separately through <code>tso_data(major =)</code>). Markers without a weight get weight 1.

Details

- "additive": VanRaden (2008) method 1, generalised to any ploidy (VanRaden 2012): $G = ZZ' / (k \sum p(1 - p))$ with $Z = X - kp$, where k is the ploidy.
- "dominance": the genotypic dominance matrix of Vitezica et al. (2013), diploids only.
- "pedigree": the numerator relationship matrix A from a pedigree (tabular method).

Allele frequencies are taken from base_freq when supplied (or stored in geno by `tso_qc()`). Pinning them to a base population keeps parent-offspring relationships meaningful across breeding cycles.

Value

A symmetric matrix with line IDs as dimnames.

Examples

```
g <- tso_qc(tso_simulate_panel(n = 40, markers = 300), verbose = FALSE)
K <- tso_kinship(g)
round(K[1:4, 1:4], 2)
# weighted: markers 1-10 count five times as much
Kw <- tso_kinship(g, weights = setNames(rep(5, 10), colnames(g$dosage)[1:10]))
```

tso_layout	<i>Lay out a training set in the field: joint genetic and field design</i>
------------	--

Description

`tso_design()` decides *which* units to phenotype; `tso_layout()` decides *where* each plot goes. It assigns the plan's training units, replicated checks and optional partially replicated (p-rep) second plots to a rows x cols field grid, in blocks, so that

Usage

```
tso_layout(
  plan,
  rows,
  cols,
  blocks = NULL,
  checks = NULL,
  check_reps = NULL,
  prep = 0,
  iterations = 20000,
  seed = 1
)
```

Arguments

plan	A tso_plan from <code>tso_design()</code> .
rows, cols	Field size: number of rows and columns of plots.
blocks	Number of blocks. Default: <code>round(sqrt(n / 10))</code> for n entries (at least 1; at least 2 with <code>prep > 0</code>).
checks	Optional check IDs. They need not be in the plan or even in the data.
check_reps	Plots per check per block (default 1: an augmented design). A single number or one value per check.
prep	Fraction of training lines given a second plot (partially replicated design), between 0 and 1.
iterations	Simulated-annealing iterations (default 20,000).
seed	Random seed (the global random state is restored afterwards).

Details

- every block receives a genetically balanced share of the lines (same mix of genetic groups), so block effects are not confounded with genetic groups;
- close relatives are not planted next to each other: simulated annealing minimises the total kinship between the lines in neighbouring plots (rook neighbours: left, right, front, back), which keeps genetic and spatial trends from being confounded;
- checks are spread systematically through every block and, where possible, never touch another check.

Field geometry. Plots are numbered in serpentine order (row 1 left to right, row 2 right to left, ...). Blocks are consecutive runs of this serpentine sequence, so each block is a contiguous band of whole or partial rows (a common layout for planters and harvesters). When the field is larger than needed, the unused plots are left at the end of the serpentine sequence (type "empty").

Allocation. Lines are grouped into strata (the plan's stratification or the group labels of the data; otherwise k-means clusters on the first principal components of the training set's relationship matrix) and dealt round-robin to the blocks within each stratum, so block sizes differ by at most one entry. Second plots (p-rep, or reps > 1 in the plan) always go to a different block from the line's other plots.

Which lines get a second plot (prep > 0): those with the highest value when the plan carries a value column in plan\$selected (the loss in CDmean when the line is dropped); otherwise those whose second plot adds most to the plan's CDmean on its targets (computed exactly with the rank-one engine; it favours genetically unique lines with few relatives in the set, whose values are least well known).

Search. Starting from a random placement within blocks, simulated annealing proposes swaps of two entries within a block, or of two entries of the same stratum in different blocks (keeping the block balance and the "replicates in different blocks" rule), with a geometric cooling schedule. Checks stay where they were placed.

Metrics. Mean kinship between neighbouring entries before (random placement) and after the search; the exact CDmean of the plan's targets with block effects fixed (block indicators and intercept projected out, every plot a record, checks included when they are genotyped units of the data), and the same without block effects. The first is never larger: the difference is the information spent on estimating the blocks.

Relation to `tso_design()`. The layout does not change which units are tested; it only places them. Lines of the plan that are also named in checks are planted as checks. Hybrid plans are laid out by hybrid, with the hybrids' relationships built from their parents. Multi-environment and multi-trait plans give an error: lay out each environment's plan separately (one call per field).

Value

A `tso_layout` object: field (data frame with one row per plot: plot, row, col, block, id, type = "training", "check", "prep" or "empty", rep, stratum), metrics, trace (objective along the search) and spec. Print it, plot it with `plot()`, and get the field book with `as.data.frame()`.

Examples

```
panel <- tso_simulate_panel(n = 150, markers = 500, seed = 1)
plan <- tso_design(panel, n = 40, verbose = FALSE)
lay <- tso_layout(plan, rows = 6, cols = 8, blocks = 2, checks = c("CHK1", "CHK2"),
                  iterations = 5000)

lay
head(as.data.frame(lay))
```

```
plot(lay)
```

tso_licence

Activate your TSOpt licence key

Description

TSOpt is free of charge for everyone, but each user needs a personal licence key from PlantNura: request one with `tso_licence_request()`. `tso_licence("<key>")` checks the key (offline: nothing is sent anywhere), activates it and saves it in your R configuration folder, so it is found automatically from then on. Without an argument it reports the current licence.

Usage

```
tso_licence(key = NULL, save = TRUE)
```

Arguments

key	The licence key (a text starting with TS01 .), or NULL to show the current licence.
save	Save the key for future sessions (default TRUE).

Details

The key is personal to the person or organisation it was issued to, and its licence ID is recorded in every plan, plan file, manifest and report you make. Keys expire (usually after 12 months); ask for a new one when yours does. On shared or automated systems (servers, CI, Docker) the key can instead be given in the environment variable TSOPT_LICENCE.

Value

The licence record (ID, name, email, organisation, country, issue and expiry dates), invisibly.

See Also

`tso_licence_request()`, and `file.show(system.file("LICENSE", package = "TSOpt"))` for the licence terms.

Examples

```
tso_licence()
```

tso_licence_request	<i>Request a free TSOpt licence key</i>
---------------------	---

Description

Writes the request email to PlantNura (and opens it in your mail program in an interactive session). PlantNura replies with your personal licence key, which you activate with `tso_licence()`. TSOpt is free of charge for everyone, including companies; registration lets PlantNura keep users informed of updates and corrections.

Usage

```
tso_licence_request(
  name = "",
  email = "",
  organisation = "",
  country = "",
  use = "",
  open = interactive()
)
```

Arguments

name, email, organisation, country	Who the licence is for.
use	One or two sentences on what you will use TSOpt for (crop, programme).
open	Open the email in your mail program (default: in interactive sessions).

Value

The email text, invisibly.

Examples

```
tso_licence_request("A. Breeder", "a.breeder@example.org", "Example Seeds", "India",
  "Training sets for our rice hybrid programme", open = FALSE)
```

tso_major_effects	<i>Major-effect loci from marker effects you already have</i>
-------------------	---

Description

The alternative to `tso_major_scan()` when no phenotypes are at hand but marker effects are: from a published GWAS or QTL study, an earlier genomic-prediction model (rrBLUP, BayesB, BGLR, ...), or known gene effects. Each marker's share of genetic variance is computed **in this panel** (its effect squared times its dosage variance here), so a large effect at an allele that is rare in your candidates is correctly given a small share. The leads are clumped as in the scan, and the result is passed to `tso_data()` as `major` = exactly like a scan.

Usage

```
tso_major_effects(
  geno,
  effects,
  var_g = NULL,
  h2 = NULL,
  var_p = NULL,
  type = c("auto", "joint", "marginal"),
  p_max = NULL,
  min_share = 0.05,
  clump_r2 = 0.5,
  window = 50,
  region_r2 = 0.2,
  max_loci = 10,
  verbose = TRUE
)
```

Arguments

geno	Genotypes: a tso_geno, a tso_data with genotypes, or anything tso_read() accepts.
effects	Marker effects: a named numeric vector (names = markers), a data frame, or the path of a CSV / tab-separated file. Recognised columns (any case): marker (marker, snp, id, name, locus), effect (effect, beta, b, alpha, estimate), optionally the effect allele (allele, effect_allele, a1) and a p-value (p, pvalue, p_value, pval). Effects are per copy of the effect allele; without an allele column, per copy of the allele the dosages count.
var_g	Genetic variance of the trait in the units of the effects.
h2, var_p	Alternatively, the heritability and phenotypic variance ($\text{var}_g = h2 * \text{var}_p$).
type	"auto", "joint" or "marginal": see Details.
p_max	For GWAS effects: largest p-value of a candidate. Default: Bonferroni (0.05 / markers) for full summary statistics (100 or more markers), no filter for a short table of reported loci.
min_share	Smallest share of genetic variance for a locus to count as major (default 0.05, 5%).
clump_r2, window	LD clumping of leads, as in tso_major_scan() .
region_r2	Markers within window of a lead and in LD with it ($r^2 \geq \text{region_r2}$) form its region for share_region.
max_loci	Maximum number of loci reported as major.
verbose	Print a summary.

Details

The genetic variance the shares are relative to is, in order: var_g ; $h2 * \text{var}_p$; or, when the effects cover most of the genome (a genome-wide model), the variance of the genomic values the effects predict in this panel (at least 100 markers, or 20% of them, have effects). With only a few effects and none of these, the shares are unknown and [tso_data\(\)](#) uses its default share (30%).

Two kinds of effects are handled (type):

- "joint": effects from one model fitted to all markers together (BayesB, BayesC, BayesR, LASSO, or the known effects of cloned genes). A model that splits one QTL over several markers in LD is handled by `share_region`, the variance of the summed effects of the markers in LD with the lead; leads are kept when it reaches `min_share`.
- "marginal": single-marker GWAS effects. They overlap through LD, so they are never summed: only markers with $p \leq p_{\max}$ are candidates, each lead's share is its own, and the genetic variance must be given (`var_g`, or `h2` and `var_p`).

"auto" (default) takes effects with a p-value column as GWAS results. GWAS effects from a small study are inflated by the winner's curse (the Beavis effect): a significant effect estimated on a few hundred lines is overestimated, so its share is an upper bound. Effects from a large study are better; so is the joint estimate of `tso_major_scan()` on your own phenotypes. Ridge effects (rrBLUP, GBLUP back-solved to markers) are a poor source: ridge regression shrinks a large effect and spreads it over thousands of markers, so it cannot size a major gene (in simulations a QTL explaining 30% of the variance received well under 1%). They are recognised (dense, near-normal effects) and a warning suggests GWAS or BayesB effects, or `tso_major_scan()` on the phenotypes.

Value

A `tso_major_effects` object (also a `tso_major_scan`) with the per-marker table, the clumped leads (`share_genetic`, `share_region`, `share_joint`) and `major_share`, the joint share of the leads. Pass it to `tso_data()` as `major =`.

See Also

`tso_major_scan()` to find loci from phenotypes.

Examples

```
g <- tso_read(system.file("extdata", "example_panel.vcf.gz", package = "TSOpt"), verbose = FALSE)
eff <- read.csv(system.file("extdata", "example_marker_effects.csv", package = "TSOpt"))
me <- tso_major_effects(g, eff)
tp <- tso_data(g, major = me, verbose = FALSE)
```

`tso_major_scan`

Find major-effect loci before designing a training set

Description

A fast genome-wide scan (EMMAX: one REML fit of the polygenic model, then a generalised least-squares test of every marker) that finds loci with large effects and estimates the share of genetic variance each one explains. Pass the result to `tso_data()` as `major =` so the design keeps every allele of these loci in the training set.

Usage

```
tso_major_scan(
  geno,
  y,
  maf = 0.05,
  n_pc = 0,
  threshold = NULL,
```

```

    clump_r2 = 0.5,
    window = 50,
    max_loci = 10,
    verbose = TRUE
  )

```

Arguments

geno	Genotypes: a tso_geno, a tso_data with genotypes, or anything tso_read() accepts.
y	Named numeric vector of phenotypes (names = line IDs), e.g. from a previous trial or the stage-1 training set.
maf	Minimum minor-allele frequency among the phenotyped lines.
n_pc	Principal components added as fixed covariates (0 by default: the kinship already models structure).
threshold	Significance threshold on the $-\log_{10}(p)$ scale. Default: Bonferroni at 5%.
clump_r2	LD threshold for clumping.
window	Clumping window in markers.
max_loci	Maximum number of loci reported as major.
verbose	Print a summary.

Details

Leads are clumped: after the most significant marker is taken, markers on the same chromosome within window markers of it, or in LD with it ($r^2 > \text{clump_r2}$), are not reported as separate loci.

Value

A tso_major_scan object: table (every marker: effect, se, $-\log_{10} p$, share of phenotypic and genetic variance), leads (the major loci), major_share (suggested total share of genetic variance at the major loci), h2, threshold, method.

References

Kang H.M. et al. (2010) Nat Genet 42: 348-354 (EMMAX). Bernardo R. (2014) Crop Sci 54: 68-75 (major genes as fixed effects).

See Also

[tso_major_effects\(\)](#) when marker effects are known and there are no phenotypes.

Examples

```

panel <- tso_simulate_panel(n = 200, markers = 500, seed = 1)
tr <- tso_simulate_trait(panel, h2 = 0.6, n_qtl = 50, major = 1,
  major_share = 0.4, seed = 2)
scan <- tso_major_scan(panel, setNames(tr$y, tr$id))
scan

```

tso_manifest	<i>A manifest of what produced a plan</i>
--------------	---

Description

MD5 fingerprints of the inputs of a plan (genotypes, relationship matrix, units, settings) with the package and R versions, the seed and the date. Two plans with the same manifest were made from the same data and settings; [tso_verify\(\)](#) checks a plan against data you have.

Usage

```
tso_manifest(plan)
```

Arguments

plan	A tso_plan.
------	-------------

Value

A named list.

tso_next_batch	<i>Choose the next batch of plots from stage-1 results (two-stage design)</i>
----------------	---

Description

After a first stage (a nursery, a preliminary yield trial, the first year of a sparse test), `tso_next_batch()` decides where the next plots do the most good *for the selection you will make*. Candidates whose ranking is uncertain near the selection threshold, and relatives that would resolve them, are worth more than candidates that are clearly in or clearly out.

Usage

```
tso_next_batch(
  plan,
  y,
  n,
  select_top = 0.1,
  pool = NULL,
  candidates = NULL,
  retest = TRUE,
  h2 = NULL,
  n_screen = 40,
  n_quad = 20,
  n_draws = 1000,
  seed = 1
)
```

Arguments

plan	The stage-1 tso_plan from <code>tso_design()</code> .
y	Named vector of stage-1 phenotypes (names = unit IDs). May also contain earlier data.
n	Number of plots in the next batch.
select_top	Fraction you will finally select (default 0.1).
pool	Units you will select among. Default: the plan's targets if it was targeted, otherwise every unit.
candidates	Units that may get a plot in the next batch. Default: every unit in the pool or the plan's candidates.
retest	Allow a second plot on units already phenotyped (as when the best lines of a preliminary trial are tested again). Default TRUE.
h2	Heritability of one plot. NULL (default) estimates it from y by maximum likelihood.
n_screen	Candidates scored exactly at each step (the rest are screened out with a fast boundary score).
n_quad	Gauss-Hermite quadrature nodes.
n_draws	Posterior draws for the probability of being selected.
seed	Random seed.

Details

This is Bayesian decision theory: the posterior of the genetic values given the stage-1 data is updated analytically, and each candidate plot is scored by its knowledge gradient: the expected increase in the mean true value of the top `select_top` fraction. Unlike the criteria used before any phenotype exists (CDmean and friends), this one depends on the data.

Value

A `tso_batch` object: the next batch (selected), the posterior for every unit in the pool with its probability of being in the top fraction (posterior), and the expected gain before and after the batch.

References

Frazier P.I., Powell W.B., Dayanik S. (2009) INFORMS J Comput 21: 599-613.

Examples

```
panel <- tso_simulate_panel(n = 200, markers = 500, seed = 1)
tr <- tso_simulate_trait(panel, h2 = 0.4, seed = 2)
stage1 <- tso_design(panel, n = 40, verbose = FALSE)
y1 <- setNames(tr$y, tr$id)[stage1$selected$id]
nb <- tso_next_batch(stage1, y1, n = 20, select_top = 0.1)
nb
```

tso_omics_kernel

*Omics kernel: transcriptome, metabolome or spectra***Description**

Builds a relationship matrix (kernel) of lines from their omics profiles, to be used next to the genomic kernel in `tso_data()` (kernel =). The design then counts the part of performance that the profiles predict. Near-infrared spectra measured on every candidate give *phenomic selection* (Rincent et al. 2018); transcriptomes and metabolomes measured on some lines give the multi-omics prediction of Riedelsheimer et al. (2012) and Schrag et al. (2018).

Usage

```
tso_omics_kernel(
  x,
  id = NULL,
  type = c("auto", "transcriptome", "metabolome", "spectra", "other"),
  batch = NULL,
  covariates = NULL,
  transform = c("auto", "none", "log2", "log"),
  spectra = c("snv_d1", "snv", "d1", "none"),
  window = 11,
  method = c("linear", "gaussian"),
  bandwidth = 1,
  top = NULL,
  max_missing = 0.2,
  outlier_z = 4,
  geno = NULL,
  fill = TRUE,
  blend = 0.05,
  name = NULL
)
```

Arguments

x	Profiles: a numeric matrix or data frame with one row per sample (line IDs as row names, or in column id), or a CSV/TSV file of the same.
id	Name of the ID column (default: "id", else row names, else the first column if it is not numeric).
type	"auto", "transcriptome", "metabolome", "spectra" or "other". "auto" recognises spectra (many columns named by wavelength whose neighbours are highly correlated).
batch	Batch of each sample (vector in row order, or the name of a column of x).
covariates	Other sample covariates to adjust for (a data frame in row order, or names of columns of x).
transform	"auto", "none", "log2" or "log".
spectra	Pre-treatment of spectra: "snv_d1" (default), "snv", "d1" or "none".
window	Savitzky-Golay window (odd number of points).
method	"linear" (default) or "gaussian".

bandwidth	Multiplier of the median distance for "gaussian".
top	Keep the top most variable features (default: all).
max_missing	Largest share of missing values a feature may have.
outlier_z	Robust z-score above which a line is flagged.
geno	Optional genotypes (tso_geno, a file or a tso_data) for the heritability and agreement checks.
fill	Predict the omics relationships of genotyped lines without a profile from their genomic relationships in <code>tso_data()</code> (default TRUE).
blend	Weight of the genomic relationships blended into the profiled block before filling (keeps it invertible; default 0.05).
name	Name of the kernel (default from type: "TX", "MET", "NIRS" or "OMICS").

Details

Steps, in order (each recorded in log):

1. **Filter:** features missing in more than max_missing of the samples and samples missing more than half their features are dropped.
2. **Transform:** "auto" takes $\log_2(x + 1)$ for count-like data (transcriptome: non-negative with values above 100) and $\log_2(x + c)$ for skewed positive data (metabolome; c half the smallest positive value); spectra and other data are not transformed.
3. **Spectra** (type = "spectra"): standard normal variate per sample and a Savitzky-Golay first derivative (window points, quadratic), the usual pre-treatment for near-infrared spectra.
4. Remaining missing values: the feature mean.
5. **Batch correction:** every feature is adjusted for batch and covariates by least squares (the feature mean is kept). Use it for sequencing runs, extraction plates, sampling dates or instruments.
6. **Replicates:** samples of the same line are averaged.
7. **Features:** constant features are dropped; top keeps the most variable ones.
8. **Kernel:** features are standardised; "linear" gives $O = ZZ'/p$, "gaussian" $\exp(-d^2/h)$ with h the median squared distance times bandwidth. O is scaled to mean diagonal 1.

Quality control: lines whose profile lies unusually far from the rest (robust z-score of the distance to the centre above outlier_z) are flagged. With geno, two checks of how genetic the profiles are: the **heritability of each feature** (REML on the genomic relationships, one decomposition for all features) and the **agreement** of the omics and genomic relationships (correlation of their off-diagonal entries). High heritability means genomics can stand in for missing profiles; low heritability means the profiles carry non-genetic signal (environment, tissue, batch) that genomics cannot predict.

Lines without a profile are not dropped by `tso_data()`: when genotypes are available, their omics relationships are predicted from the genomic ones (single-step, see *Details* of the source; fill = FALSE drops them instead). The prediction is only as good as the feature heritabilities.

Value

A tso_omics object: K (the kernel of the profiled lines), name, type, lines, features, qc (per line: distance z-score, outlier flag), h2 (feature heritabilities, with geno), agreement, log. Pass it in kernels = of `tso_data()`, alone (the genomic kernel is added) or with other kernels: `tso_data(geno, kernels = list("A", NIRS = om))`.

References

Rincent R et al. (2018) G3 8:3961-3972. Riedelsheimer C et al. (2012) Nat Genet 44:217-220. Schrag TA et al. (2018) Genetics 208:1373-1385. Legarra A, Aguilar I, Misztal I (2009) J Dairy Sci 92:4656-4663.

See Also

[tso_omics_plan\(\)](#) for which lines to profile; [tso_variance\(\)](#) (kernels =) to estimate the omics share from phenotypes.

Examples

```
panel <- tso_simulate_panel(n = 100, markers = 400, seed = 1)
om <- tso_simulate_omics(panel, features = 200, seed = 1)
ok <- tso_omics_kernel(om$profiles, geno = panel)
ok
tp <- tso_data(panel, kernels = ok, verbose = FALSE)
```

tso_omics_plan

Plan omics profiling and phenotyping under one budget

Description

Profiles cost money that could buy plots. On a pilot set of lines that are genotyped and profiled, `tso_omics_plan()` evaluates every split of one budget between profiling (`n_profile` lines at `profile_cost`) and phenotyping plots (`plot_cost`):

1. the lines to profile are chosen (strategy) by genomic coverage ([tso_genotype_set\(\)](#)); as the lines to be phenotyped (a design on the genomic kernel, topped up by coverage); as the lines to be predicted (target, or the lines such a design leaves unphenotyped; by coverage among them, topped up by coverage); or at random;
2. their profiles are kept and the omics relationships of all other lines are predicted from genomics, as [tso_data\(\)](#) does;
3. [tso_design\(\)](#) chooses the plots on that model (genomic + omics kernel, omics share share);
4. the plan is scored by its **realised reliability** under the true omics relationships (the pilot's full profiles): the squared correlation of predicted and true values of the targets, as in [tso_imputation_plan\(\)](#). It equals CDmean when the model is the truth.

Usage

```
tso_omics_plan(
  geno,
  omics,
  budget,
  profile_cost,
  plot_cost,
  n_profile = NULL,
  share = 0.3,
  h2 = 0.5,
  strategy = c("coverage", "phenotyped"),
  target = NULL,
  seed = 1
)
```

Arguments

geno	Genotypes of the pilot lines (tso_geno, a file or a tso_data).
omics	A <code>tso_omics_kernel()</code> of the same lines (all profiled).
budget	Total budget, in the units of the costs.
profile_cost, plot_cost	Cost of one omics profile and of one phenotyping plot.
n_profile	Numbers of lines to profile (default 25%, 50%, 75%).
share	Share of the omics kernel in the genetic + omics variance (e.g. from <code>tso_variance()</code>).
h2	Heritability (signal over signal plus noise).
strategy	One or more of "coverage", "phenotyped", "targets", "random".
target	Lines to predict (default: the lines not phenotyped).
seed	Random seed.

Details

Two plans are added for comparison: no profiling (every coin on plots, genomic kernel only) and profiling every line. The table also reports the **fidelity** of the predicted omics relationships of un-profiled lines (correlation with the true ones).

Value

A `tso_omics_plan` object: table (plan, strategy, lines profiled, cost, plots, fidelity, expected and realised reliability), best, settings.

Examples

```
panel <- tso_simulate_panel(n = 120, markers = 400, seed = 1)
om <- tso_simulate_omics(panel, features = 150, seed = 1)
ok <- tso_omics_kernel(om$profiles)
op <- tso_omics_plan(panel, ok, budget = 3000, profile_cost = 20, plot_cost = 30,
  n_profile = c(30, 60))
op
```

tso_pareto

Multi-objective training-set design: the Pareto front

Description

`tso_design()` optimises one criterion. Real decisions trade several off: how reliable the predictions will be, how diverse (unrelated) the training set is, what it costs, how well it covers the targets. `tso_pareto()` searches for all of them at once and returns the **Pareto front**: every training set that no other set beats on all objectives at the same time. Look at the trade-off with `plot()`, then take one set with `tso_pareto_pick()` (by default the "knee", where giving up a little of one objective stops buying much of the other).

Usage

```

tso_pareto(
  data,
  n,
  objectives = c("cdmean", "coancestry"),
  target = NULL,
  h2 = 0.5,
  candidates = NULL,
  must_include = NULL,
  exclude = NULL,
  pop = 100,
  generations = 150,
  seed = 1,
  verbose = TRUE,
  directions = NULL
)

## S3 method for class 'tso_pareto'
print(x, ...)

## S3 method for class 'tso_pareto'
plot(x, ...)

```

Arguments

<code>data</code>	A <code>tso_data</code> object, or anything <code>tso_data()</code> accepts.
<code>n</code>	Number of units in the training set (including <code>must_include</code>).
<code>objectives</code>	Objectives to trade off: a character vector of built-in names (see the table), or a list mixing names and functions <code>f(ids, data)</code> . At least two.
<code>target</code>	Units you want to predict (IDs). <code>NULL</code> : the whole pool.
<code>h2</code>	Assumed entry-mean heritability (one value in $(0, 1]$); the noise variance is $(1 - h2) / h2$ on the genetic-variance scale.
<code>candidates</code>	Units that may be phenotyped. Default: every unit that is not a target (for hybrids, only crosses between crossable parents).
<code>must_include</code>	Units that must be in every set; they count towards <code>n</code> .
<code>exclude</code>	Units that can never be in the set.
<code>pop</code>	Population size of the genetic algorithm (and the maximum number of sets on the returned front).
<code>generations</code>	Number of NSGA-II generations.
<code>seed</code>	Random seed.
<code>verbose</code>	Print progress messages.
<code>directions</code>	Optional named character vector, "max" or "min" per objective, e.g. <code>c(custom = "min")</code> . Functions default to "max".
<code>x</code>	A <code>tso_pareto</code> object.
<code>...</code>	For <code>plot()</code> : style (see <code>tso_theme()</code>) and axes (the two objectives to show, names or positions; default the first two).

Details

Built-in objectives (reported on their natural scale):

objective	what	
"cdmean"	CDmean with the intercept as an exact fixed effect (Rincent et al. 2012)	dire
"cdmin"	worst-case CD: the least reliable target	max
"coancestry"	mean off-diagonal relationship (K) among the training units	max
"cost"	total cost of the set (cost in <code>tso_data()</code>)	min
"dopt"	D-optimality, $\log \det(I + K_{SS} / s^2)$	min
"coverage"	mean over targets of the closest training relative (correlation scale)	max
"gain"	expected mean genetic value of the top 10% selected on GEBVs	max
"haplotypes"	share of the targets' haplotype copies whose haplotype is in the training set (<code>tso_haplotypes()</code>)	max

Your own objectives are functions $f(ids, data)$ of the selected unit IDs (including `must_include`) and the `tso_data` object, returning one number. They are maximised unless directions (or an attribute "direction" on the function) says "min". Mix them freely with names: `objectives = list("cdmean", spread = my_fun)`.

How it works. NSGA-II (Deb et al. 2002) over subsets of fixed size: fast non-dominated sorting, crowding distance, binary tournaments on (front, crowding), a subset crossover that keeps the units both parents share and fills the rest from the units only one parent has, and swap mutation. The first population is seeded with the greedy CDmean set from the rank-one engine and with greedy solutions of weighted sums of the built-in objectives, then filled with random sets. An external archive keeps the non-dominated sets among every set ever evaluated (capped at pop by crowding distance, extremes always kept), so the best CDmean on the front is never below the greedy CDmean set.

Value

A `tso_pareto` object, a list with

- `front`: a data frame, one row per non-dominated set, with `id` (the solution number, the index into sets) and one column per objective on its natural scale, sorted by the first objective;
- `sets`: a list of unit-ID vectors, one per row of `front`;
- `knee`: the id of the knee solution;
- `seeds`: the objective values of the greedy CDmean set;
- `all_evaluated`: the final population (objectives and front rank);
- `spec`: settings, objective directions, the normalised 2-D hypervolume (hypervolume, with its bounds `hv_lo` / `hv_hi`) and the number of sets evaluated;
- `time`: elapsed seconds.

References

Deb K., Pratap A., Agarwal S., Meyarivan T. (2002) IEEE Trans Evol Comput 6: 182-197. Rincent R. et al. (2012) Genetics 192: 715-728. Akdemir D., Rio S., Isidro y Sanchez J. (2021) TrainSel: an R package for selection of training populations. Front Genet 12: 655287.

See Also

`tso_pareto_pick()`, `tso_design()`

Examples

```

panel <- tso_simulate_panel(n = 150, markers = 500, seed = 1)
fr <- tso_pareto(panel, n = 30, pop = 40, generations = 30, verbose = FALSE)
fr
plot(fr)
chosen <- tso_pareto_pick(fr)           # the knee
best_cd <- tso_pareto_pick(fr, "best_cdmean")
# a user objective: the closest pair of relatives in the set (minimised)
maxrel <- function(ids, data) { k <- data$K[ids, ids]; max(k[upper.tri(k)]) }
fr2 <- tso_pareto(panel, n = 30, objectives = list("cdmean", maxrel = maxrel),
                  directions = c(maxrel = "min"), pop = 30, generations = 20,
                  verbose = FALSE)

fr2$front

```

tso_pareto_pick	<i>Take one training set from a Pareto front</i>
-----------------	--

Description

Take one training set from a Pareto front

Usage

```
tso_pareto_pick(x, which = "knee", weights = NULL)
```

Arguments

x	A tso_pareto object from tso_pareto() .
which	"knee" (default): the balanced compromise (see tso_pareto()); "best_<objective>", e.g. "best_cdmean": the set that is best on that objective; or a solution number (x\$front\$id).
weights	Optional named numeric weights, one per objective, applied to the objectives after scaling each to 0 (worst on the front) - 1 (best); the set with the highest weighted sum is returned. Overrides which. Unnamed objectives get weight 0.

Value

A character vector of unit IDs (the training set, must_include first), with attribute "solution" (the row id in x\$front) and "values" (its objective values).

Examples

```

panel <- tso_simulate_panel(n = 120, markers = 400, seed = 1)
fr <- tso_pareto(panel, n = 25, pop = 30, generations = 20, verbose = FALSE)
tso_pareto_pick(fr)
tso_pareto_pick(fr, "best_coancestry")
tso_pareto_pick(fr, weights = c(cdmean = 2, coancestry = 1))

```

tso_plan_json	<i>Write a plan as a versioned JSON file</i>
---------------	--

Description

A stable, documented representation of a plan for other software (pipelines, databases, LIMS): schema `tsopt-plan/1` with the manifest (`tso_manifest()`), the settings, the selected units and the targets, and the expected values. Needs the `jsonlite` package.

Usage

```
tso_plan_json(plan, file = NULL)
```

Arguments

plan	A <code>tso_plan</code> .
file	Output file (<code>.json</code>); <code>NULL</code> returns the JSON text.

Details

Fields: `manifest`, `spec` (system, criterion, h2 or its grid, size or budget, value, search, fixed effects), `selected` (order, id, line, env, trait, forced, value), `targets` (ids), `value` (cdmean, cdmean_exact, pevmean, random_mean, gain_accuracy), `verdict`.

Value

The JSON text, invisibly.

tso_plot	<i>Publication-ready figures for every TSOpt object</i>
----------	---

Description

`tso_plot()` draws any figure in the TSOpt gallery and returns an ordinary `ggplot2` object, so you can restyle, combine or annotate it. Save it at journal size with `tso_save_figure()`.

Usage

```
tso_plot(x, type = NULL, style = "publication", title = NULL, ...)
```

Arguments

x	A result object from TSOpt: a plan, diagnosis, comparison, evaluation, data, batch, simulation, Pareto front, sample-size curve, top-k analysis, contribution plan or major-locus scan.
type	Figure type (see table). <code>NULL</code> gives the default for x.
style	Figure style passed to <code>tso_theme()</code> .
title	Optional title; <code>NULL</code> keeps the automatic one, <code>FALSE</code> drops it.
...	Passed to the individual figure functions.

Details

object	type
plan	"summary" (default)
plan	"pca"
plan	"divide"
plan	"coverage"
plan	"baseline"
plan	"search"
plan	"kinship"
plan (hybrid)	"hybrid_grid"
plan (environments)	"met_allocation"
plan (traits)	"trait_allocation"
diagnosis	"summary", "size_curve", "structure", "baseline"
comparison	"compare"
data	"pca", "kinship"
batch	(none)
simulation	(none)
plan	"major"
Pareto front	(none)
sample size	(none)
top-k	(none)
contributions	(none)
major scan	(none)
major effects	(none)
omics kernel	(none)
omics plan	(none)
large plan	(none)

shows
four panels A-D: PCA, divide, coverage
where the training set sits in genetic space
reliability of each candidate (the predicted
closest training relative of each target
the plan against random training sets
criterion value along the search
relationship heatmap ordered by group,
which crosses were chosen; H0/H1/H2
line x environment allocation
which trait is measured on which line
see [tso_diagnose\(\)](#)
methods side by side, see [tso_compare\(\)](#)
the panel before any design
posterior ranking and where the next plot
gain, genetic variance and accuracy over
carriers of each major-locus allele: target
trade-off between objectives, see [tso_plot\(\)](#)
operating curve, see [tso_sample_size\(\)](#)
chance of finding the true best targets, see
optimum contributions vs EBV, see [tso_optimize\(\)](#)
Manhattan plot, see [tso_major_scan\(\)](#)
variance explained per marker, see [tso_variance\(\)](#)
profiles by principal components, feature
realised reliability by lines profiled, see
search trace, see [tso_design_large\(\)](#)

Value

A ggplot object (or a patchwork object for "summary").

Examples

```
panel <- tso_simulate_panel(n = 150, markers = 500, seed = 1)
plan <- tso_design(panel, n = 40, verbose = FALSE)
tso_plot(plan, "pca")
```

tso_qc	<i>Quality control and imputation of genotypes</i>
--------	--

Description

Removes poor markers and lines, reports heterozygosity, imputes the remaining missing values and stores base-population allele frequencies. Every step is recorded in the object's log.

Usage

```
tso_qc(
  geno,
  maf = 0.05,
  call_rate = 0.8,
  line_call_rate = 0.5,
  impute = c("mean", "none"),
  base_freq = NULL,
  verbose = TRUE
)
```

Arguments

geno	A tso_genotype object (or anything <code>tso_read()</code> accepts).
maf	Minimum minor-allele frequency (default 0.05).
call_rate	Minimum call rate per marker (default 0.8).
line_call_rate	Minimum call rate per line (default 0.5).
impute	"mean" (default; adequate for relationship matrices) or "none".
base_freq	Allele frequencies of the base population, named by marker. When given they are kept for all later relationship matrices (so relationships stay comparable across breeding cycles). When NULL, frequencies are estimated from this panel and stored.
verbose	Print what was done.

Value

The filtered tso_genotype object, with base_freq added.

Examples

```
g <- tso_qc(tso_simulate_panel(n = 60, markers = 300))
```

tso_read	<i>Read genotypes from any common format</i>
----------	--

Description

`tso_read()` reads a genotype file (or an R object) and converts it to one internal form: a lines x markers matrix of allele dosages (0 to ploidy). The format is detected from the file content as well as its extension, and compressed files (.gz, .bz2, .xz, .zip) are handled transparently.

Usage

```
tso_read(
  x,
  format = "auto",
  ploidy = NULL,
  markers_in = c("auto", "columns", "rows"),
  id_col = NULL,
```

```
vcf_field = c("GT", "DS"),
binary_as = c("homozygous", "dosage"),
verbose = TRUE
)
```

Arguments

x	A file path (or PLINK prefix), a matrix or a data frame.
format	"auto" (default) or one of "vcf", "hapmap", "plink", "ped", "raw", "dart", "table", "excel", "rds".
ploidy	Ploidy. Detected from the data when NULL.
markers_in	For tables only: "auto", "columns" or "rows". With "auto", a table with more rows than columns is assumed to have markers in rows (genotype panels almost always have more markers than lines).
id_col	For tables only: the column holding line IDs. Detected when NULL (a first column that is not genotype data).
vcf_field	For VCF only: "GT" (default) or "DS" (dosage).
binary_as	For tables coded only 0/1: "homozygous" (default; 0/1 is read as 0/2, the usual coding for inbred lines) or "dosage".
verbose	Print the read report.

Details

Supported inputs:

Format	How it is recognised
VCF (.vcf, .vcf.gz)	##fileformat=VCF header; reads GT (or DS)
HapMap (.hmp.txt)	rs#, alleles, chrom, pos columns
PLINK binary (.bed/.bim/.fam)	a file prefix, or any of the three files
PLINK text (.ped/.map)	.ped extension
PLINK additive (.raw)	FID IID PAT MAT SEX PHENOTYPE header
DArTseq one-row / two-row	AlleleID column after * header rows
Delimited tables (.csv, .tsv, .txt)	anything else that is text
Excel (.xlsx, .xls)	extension (needs readxl)
R files (.rds, .RData)	extension
R objects	a matrix or data frame

Tables may use any coding: 0/1/2, -1/0/1, 0/1 (homozygous lines), 0 ... ploidy dosages, AA/AB/BB, nucleotide pairs (AG, A/G, A|G) or single IUPAC letters. Missing-value codes such as NA, N, -, --, ., ./., -9 and -1 (Axiom) are recognised.

Value

A `tso geno` object: a list with dosage (lines x markers matrix), map (marker table), ploidy, format, source and log.

See Also

[tso_qc\(\)](#), [tso_data\(\)](#), [tso_write\(\)](#)

Examples

```
g <- tso_simulate_panel(n = 50, markers = 200)
f <- tempfile(fileext = ".csv")
tso_write(g, f)
g2 <- tso_read(f)
g2
```

tso_report

*One-page report for a programme manager***Description**

Writes a self-contained HTML page with the decision (how many units, chosen how), the expected gain over random sampling, the risks (coverage, the predictability divide, structure) and the figures, all produced by the same functions you can call yourself. No pandoc or LaTeX is needed.

Usage

```
tso_report(
  plan,
  file = "tso_report.html",
  diagnosis = NULL,
  comparison = NULL,
  title = "Training-set plan",
  author = NULL
)
```

Arguments

plan	A tso_plan.
file	Output HTML file.
diagnosis	Optional tso_diagnosis to include (size curve and structure).
comparison	Optional tso_comparison to include; computed when NULL.
title	Report title.
author	Author line.

Value

The file name, invisibly.

Examples

```
panel <- tso_simulate_panel(n = 150, markers = 500, seed = 1)
plan <- tso_design(panel, n = 30, verbose = FALSE)
tso_report(plan, file.path(tempdir(), "plan.html"))
```

tso_sample_size	<i>How many units do I need to phenotype?</i>
-----------------	---

Description

tso_sample_size() draws the operating curve of a training set: the expected accuracy of genomic prediction at every training-set size, for an optimised design and for random sampling, expressed relative to phenotyping every candidate. It answers "phenotype how many lines to reach 95% of the accuracy I would get by phenotyping them all?".

Usage

```
tso_sample_size(
  data,
  target = NULL,
  sizes = NULL,
  h2 = NULL,
  relative_to = c("all", "max"),
  n_random = 20,
  seed = 1
)
```

Arguments

data	A tso_data object or anything tso_data() accepts.
target	Optional target units (IDs). Default: untargeted (the whole pool is both candidates and targets).
sizes	Optional training-set sizes at which random sampling is evaluated. The optimised curve is computed at every size up to max(sizes), or up to the whole candidate pool when sizes is NULL.
h2	Assumed heritability. NULL (default): average over h2 = 0.2, 0.5 and 0.8. One or several values in (0, 1].
relative_to	"all" (default): relative to phenotyping every candidate; "max": relative to the largest size on the curve.
n_random	Random training sets drawn at each size.
seed	Random seed.

Details

How it works. One greedy pass of the CDmean engine (the same rank-one updates as [tso_design\(\)](#), with the exact fixed-intercept form) adds the candidates one at a time, which gives the optimised CDmean at *every* size in a single run: the curve is deterministic and exact for the greedy design, not a model fitted to noisy random maxima. With h2 = NULL the curve is the robust CDmean averaged over h2 = 0.2, 0.5 and 0.8, the default of [tso_design\(\)](#). Expected accuracy is $\sqrt{CD}$, and the relative efficiency is

$$RE(n) = \sqrt{CD(n)} / \sqrt{CD_{ref}},$$

where CD_{ref} is the CDmean with every candidate phenotyped (relative_to = "all") or the largest value on the curve (relative_to = "max").

The sizes that reach 90%, 95% and 99% relative efficiency are read off the exact greedy curve (the smallest integer size whose RE reaches the level). Random sampling is evaluated at a grid of sizes (the mean of n_{random} random sets; accuracy is the mean of $\sqrt{CD}$ over the sets) and the sizes it needs are interpolated linearly between grid points. The logistic growth curve of TSDFGS, $acc(n) = \alpha / (1 + e^{\beta - \gamma n})$, is also fitted to the optimised accuracy for smoothing and reporting only; the answers never depend on it.

Value

A `tso_sample_size` object: curve (data frame with columns `n`, `cd_opt`, `acc_opt`, `re_opt`, `cd_random`, `acc_random`, `re_random`; random columns are NA at sizes not evaluated), `n_for` (smallest optimised size reaching RE 0.90, 0.95, 0.99), `n_for_random` (the same for random sampling, interpolated), `savings` (plots saved by the optimised design), `reference` (the reference CDmean and accuracy), `fit` (logistic fit or NULL) and `spec`.

References

Ou J.-H., Liao C.-T. (2019) Theor Appl Genet 132: 3151-3161 (TSDFGS). Rincent R. et al. (2012) Genetics 192: 715-728.

Examples

```
panel <- tso_simulate_panel(n = 150, markers = 400, seed = 1)
ss <- tso_sample_size(panel, n_random = 5)
ss

plot(ss)
```

tso_save_figure

Save a figure ready for a journal, a slide or a report

Description

Writes a ggplot at the exact size a journal asks for, in one or several formats, with fonts embedded and high-resolution raster output.

Usage

```
tso_save_figure(
  plot,
  file,
  journal = "generic",
  width = "single",
  height = NULL,
  format = NULL,
  dpi = 600
)
```

Arguments

plot	A ggplot (or patchwork) object.
file	File name. With an extension, that format is written; without one, every format in format is written.
journal	A journal preset (see table).
width	"single", "onehalf", "double" or a number in mm.
height	Height in mm (default: 0.72 x width, or 0.85 x width for multi-panel figures).
format	One or more of "pdf", "svg", "eps", "png", "tiff". Default: c("pdf", "png").
dpi	Resolution for raster formats (default 600).

Details

Journal presets set the column widths (mm):

journal	single	1.5 column	double
"generic"	85	120	180
"nature"	89	120	183
"plos"	132	150	190
"g3", "genetics"	85	120	178
"tag" (Theor Appl Genet), "springer"	84	129	174
"frontiers"	85	120	180
"plant_genome", "crop_science"	85	120	178
"elsevier"	90	140	190

Value

The file names written, invisibly.

Examples

```
panel <- tso_simulate_panel(n = 120, markers = 400, seed = 1)
plan <- tso_design(panel, n = 30, verbose = FALSE)
tso_save_figure(tso_plot(plan, "pca"), file.path(tempdir(), "fig1"),
               journal = "g3", width = "single")
```

tso_score	Score any training set
-----------	------------------------

Description

Computes design criteria for a given training set, the way STPGA's CDMEAN() or TSDFGS's cd_score() do, but on the TSOpt data object and with heritability made explicit.

Usage

```
tso_score(
  data,
  train,
  target = NULL,
  h2 = 0.5,
  criteria = c("cdmean", "cdmean_exact", "pevmean", "dopt"),
  value = c("total", "breeding")
)
```

Arguments

data	A tso_data object (or anything <code>tso_data()</code> accepts).
train	Training units (IDs or indices). May repeat (replicates).
target	Target units; NULL = all units not in train... for untargeted scoring, all units.
h2	Assumed entry-mean heritability.
criteria	Any of "cdmean", "cdmean_exact" (intercept projected out), "pevmean", "dopt", "rscore".
value	"total" (default) or "breeding": score the reliability of total genetic values or of breeding values (the additive part), for data with non-additive kernels (<code>tso_data(kernels =)</code>).

Value

A named numeric vector.

Examples

```
panel <- tso_simulate_panel(n = 100, markers = 400, seed = 1)
tp <- tso_data(panel, verbose = FALSE)
tso_score(tp, train = 1:30)
```

tso_season

Plan a season across stages: where the plots should go

Description

Evaluates two-stage allocations of a plot budget by simulation on your panel. In each simulated season:

1. **Stage 1:** n_1 candidates are phenotyped at `sites1` site(s) with `reps1` replicate(s). They are chosen by `tso_design()` (or at random, `stage1 = "random"`).
2. GBLUP on the stage-1 data ranks every candidate; the best k_1 advance.
3. **Stage 2:** the k_1 lines are tested at `sites2` sites with `reps2` replicate(s).
4. The final `n_select` lines are chosen by GBLUP on all the data.

Usage

```
tso_season(
  data,
  budget,
  n1 = NULL,
  k1 = NULL,
  sites2 = 1:4,
  sites1 = 1,
  reps1 = 1,
  reps2 = 1,
  n_select = 5,
  h2_plot = 0.3,
  ge_ratio = 0.5,
  plot_cost = 1,
  stage1 = c("tsopt", "random", "both"),
  reps = 50,
  seed = 1,
  verbose = TRUE
)
```

Arguments

data	A tso_data object (line level) or genotypes.
budget	Plots available (in units of plot_cost).
n1, k1, sites2	Values to try for the stage-1 training-set size, the lines advanced, and the stage-2 sites (all combinations that fit the budget are evaluated). Defaults cover a useful range.
sites1, reps1, reps2	Stage-1 sites and replicates, stage-2 replicates.
n_select	Lines selected at the end (varieties or new parents).
h2_plot	Heritability of a single plot.
ge_ratio	Genotype x environment variance relative to the genetic variance.
plot_cost	Cost of one plot (the budget is in the same units).
stage1	"tsopt" (default), "random", or "both": TSOpt and random stage-1 training sets on the same simulated seasons, with the paired difference (what the design adds within a season).
reps	Simulated seasons per allocation.
seed	Random seed.
verbose	Print progress.

Details

Genetic values are drawn from $N(0, K)$ (so gain is in genetic standard deviations); a line at one site has genotype x environment variance `ge_ratio` (relative to the genetic variance) and plot error set so that the heritability of one plot is `h2_plot`. The same simulated truths are used for every allocation (common random numbers), so allocations are compared on equal terms. BLUP uses the true variance components: this is a planning tool, not an analysis.

Value

A `tso_season` object: table (one row per allocation: plots per stage, cost, expected gain in genetic SD with its standard error, the share of the true top `n_select` lines among those selected, and the accuracy of the stage-1 prediction), best (the allocation with the highest expected gain) and settings.

Examples

```
panel <- tso_simulate_panel(n = 150, markers = 400, seed = 1)
plan <- tso_season(panel, budget = 150, n1 = c(40, 80), k1 = c(10, 20), sites2 = 2:3,
                  reps = 5, verbose = FALSE)
plan
```

<code>tso_simulate</code>	<i>Simulate recurrent genomic selection under different training-set strategies</i>
---------------------------	---

Description

A small, self-contained breeding simulator (no extra packages). Each cycle: parents are crossed, doubled haploids are made (with recombination along the chromosomes), a training set of `n_train` candidates is chosen by each strategy and phenotyped, GBLUP is trained on the phenotyped lines of the last window cycles (with allele frequencies pinned to the founders), and the best `n_parents` candidates by GEBV become the next parents.

Usage

```
tso_simulate(
  strategies = list(random = "random", cdmean = "cdmean", gain = "gain"),
  cycles = 6,
  reps = 2,
  n_candidates = 200,
  n_crosses = 20,
  n_train = 80,
  n_parents = 20,
  n_founders = 30,
  markers = 1000,
  n_qtl = 200,
  chromosomes = 10,
  h2 = 0.4,
  window = Inf,
  seed = 1,
  verbose = TRUE
)
```

Arguments

<code>strategies</code>	A named list: each element is a strategy name or a function (see Details).
<code>cycles</code>	Number of selection cycles.
<code>reps</code>	Number of independent replicates.

n_candidates	Doubled haploids per cycle.
n_crosses	Crosses per cycle.
n_train	Candidates phenotyped per cycle.
n_parents	Parents selected per cycle.
n_founders	Founder lines.
markers	Number of genotyped markers.
n_qtl	Number of QTL (not genotyped).
chromosomes	Number of chromosomes.
h2	Heritability of one plot in the founder generation. The error variance is then held fixed, so heritability falls as selection uses up genetic variance, as in a real programme.
window	Cycles of training data kept (Inf = cumulative).
seed	Random seed.
verbose	Print progress.

Details

All strategies start from the same founders and QTL in each replicate (common random numbers), so differences between them are due to the strategy alone.

Built-in strategies: "random", "cdmean" (untargeted CDmean over this cycle's candidates, earlier cycles as history), "gain" (expected gain at the selection intensity actually used) and "cdmean_robust" (CDmean averaged over $h^2 = 0.2, 0.5, 0.8$). You can also pass a function `f(data, n, candidates, history)` returning the IDs to phenotype.

Value

A `tso_simulation` object with a results data frame (one row per replicate x strategy x cycle). Plot it with `tso_plot()`.

Examples

```
sim <- tso_simulate(cycles = 3, reps = 1, n_candidates = 120, n_train = 40,
                  markers = 400, verbose = FALSE)
sim
```

tso_simulate_cycles	<i>Simulate a breeding programme's historical data, cycle by cycle</i>
---------------------	--

Description

Produces the kind of data a programme accumulates over the years: every cycle, crosses among the previous cycle's best lines give doubled haploids, all of which are genotyped and phenotyped in that cycle's trial (with a year effect). Parents are chosen on phenotype, so the history is not shaped by any training-set method. Use it to try `tso_backtest()` before running it on your own data.

Usage

```
tso_simulate_cycles(
  cycles = 5,
  n_candidates = 200,
  n_crosses = 20,
  n_parents = 20,
  n_founders = 30,
  markers = 1000,
  n_qtl = 200,
  chromosomes = 10,
  h2 = 0.4,
  year_sd = 1,
  seed = NULL
)
```

Arguments

<code>cycles</code>	Number of cycles (years, cohorts).
<code>n_candidates</code>	Lines per cycle.
<code>n_crosses</code>	Crosses per cycle.
<code>n_parents</code>	Lines selected as parents for the next cycle.
<code>n_founders</code>	Founder lines.
<code>markers, n_qtl</code>	Genotyped markers and (ungenotyped) QTL.
<code>chromosomes</code>	Number of chromosomes.
<code>h2</code>	Heritability of one line's phenotype in the founder generation.
<code>year_sd</code>	Standard deviation of the year effects, in genetic SD.
<code>seed</code>	Random seed.

Value

A `tso_cycles` object: `geno` (a `tso_geno` of every line, with a map), `pheno` (a data frame with `id`, `cycle`, `family`, `y` and the true genetic value `g`).

Examples

```
hist <- tso_simulate_cycles(cycles = 4, n_candidates = 80, markers = 400, seed = 1)
hist
```

<code>tso_simulate_omics</code>	<i>Simulate omics profiles and a phenotype that depends on them</i>
---------------------------------	---

Description

For testing and teaching: each feature is a heritable trait of the panel (heritabilities drawn uniformly in `h2_features`) with its own non-genetic part; the phenotype has a direct genetic part and a part mediated by the features, in the proportion `omics_share`.

Usage

```
tso_simulate_omics(
  panel,
  features = 500,
  h2_features = c(0.1, 0.7),
  omics_share = 0.4,
  h2 = 0.5,
  causal = 0.2,
  type = c("other", "spectra"),
  seed = NULL
)
```

Arguments

panel	A tso geno (e.g. from tso_simulate_panel()).
features	Number of features.
h2_features	Range of feature heritabilities.
omics_share	Share of the phenotype's signal that runs through the profiles.
h2	Heritability of the phenotype (signal over signal plus noise).
causal	Share of features that affect the phenotype.
type	"other" (a matrix of standardised features) or "spectra" (curves over 400 wavelengths: every feature an absorption band, plus light scatter, baseline offsets and instrument noise).
seed	Random seed.

Value

A list: profiles (lines x features, row names = line IDs), pheno (data frame id, y), truth (the signal: direct genetic, omics-mediated and total).

Examples

```
panel <- tso_simulate_panel(n = 80, markers = 300, seed = 1)
om <- tso_simulate_omics(panel, features = 100, seed = 1)
dim(om$profiles)
```

tso_simulate_panel	<i>Simulate a breeding panel for practice and testing</i>
--------------------	---

Description

Creates a structured panel of doubled-haploid (or outbred) lines: several subpopulations with differentiated allele frequencies (Balding-Nichols model), and within each, lines derived from crosses among founders with recombination along chromosomes. The result behaves like a real breeding panel: family structure, linkage disequilibrium and population structure.

Usage

```
tso_simulate_panel(
  n = 400,
  markers = 1500,
  groups = 4,
  fst = 0.15,
  founders = 10,
  chromosomes = 10,
  type = c("dh", "outbred"),
  seed = NULL
)
```

Arguments

n	Number of lines.
markers	Number of markers.
groups	Number of subpopulations.
fst	Differentiation between subpopulations.
founders	Founders per subpopulation.
chromosomes	Number of chromosomes (markers are spread evenly).
type	"dh" (doubled haploids, dosages 0/2) or "outbred".
seed	Random seed.

Value

A tso_geno object with an extra element groups.

Examples

```
panel <- tso_simulate_panel(n = 200, markers = 800, seed = 1)
table(panel$groups)
```

tso_simulate_trait	<i>Simulate a trait for a panel</i>
--------------------	-------------------------------------

Description

Draws true genetic values from `n_qtl` randomly placed QTL and adds noise so that the entry-mean heritability is `h2`.

Usage

```
tso_simulate_trait(
  x,
  h2 = 0.5,
  n_qtl = 100,
  major = 0,
  major_share = 0.3,
  seed = NULL
)
```

Arguments

x	A tso_geno or tso_data object.
h2	Entry-mean heritability.
n_qtl	Number of polygenic QTL (use a small number for an oligogenic trait).
major	Number of major QTL, or the names of the markers to use as major QTL (default 0: none). Randomly chosen major QTL have a minor-allele frequency of at least 0.1.
major_share	Share of genetic variance explained by the major QTL together (default 0.3).
seed	Random seed.

Value

A data frame with columns id, g (true genetic value, variance

1. and y (phenotype). With major QTL, the attribute major names them.

Examples

```
panel <- tso_simulate_panel(n = 100, markers = 400, seed = 1)
tr <- tso_simulate_trait(panel, h2 = 0.5)
head(tr)
# a trait with one major gene explaining 40% of the genetic variance
tr2 <- tso_simulate_trait(panel, h2 = 0.5, major = 1, major_share = 0.4)
attr(tr2, "major")
```

tso_theme

The TSOpt figure theme

Description

A clean ggplot2 theme sized for journal figures: white background, light horizontal grid only, left-aligned titles that state the result, and text that stays legible when the figure is printed at column width.

Usage

```
tso_theme(
  style = c("publication", "presentation", "course"),
  base_size = NULL,
  base_family = ""
)
```

Arguments

style	"publication" (default; 8 pt base text, for figures saved at 85-180 mm), "presentation" (larger text and lines for slides) or "course" (PlantNura teaching style).
base_size	Override the base text size (points).
base_family	Font family (default: the device's sans serif).

Value

A ggplot2 theme.

Examples

```
library(ggplot2)
ggplot(mtcars, aes(wt, mpg)) + geom_point() + tso_theme()
```

tso_topk	<i>Will this training set find the best candidates?</i>
----------	---

Description

tso_topk() measures what a breeder actually does with the predictions: select the top fraction. It simulates the genomic model behind the design and reports how well GBLUP trained on the training set identifies the truly best evaluation units, for the plan and for random training sets of the same size.

Usage

```
tso_topk(
  x,
  train = NULL,
  target = NULL,
  top = 0.1,
  n_sim = 500,
  h2 = NULL,
  n_random = 20,
  seed = 1
)
```

Arguments

x	A tso_plan from tso_design() , or a tso_data object (or anything tso_data() accepts) together with train.
train	Training units (IDs), when x is not a plan.
target	Units to rank (IDs). Default: the plan's evaluation units (its targets not in training, or every unit not in training when untargeted); for a tso_data input, every unit not in train.
top	Fraction selected (default 0.1).
n_sim	Number of simulated draws for the training set.
h2	Heritability of the simulated trait. Default: the plan's (median) design heritability, or 0.5 for a tso_data input.
n_random	Number of random training sets of the same size (0 to skip).
seed	Random seed.

Details

Model. Genetic values $g \sim N(0, K)$ are drawn jointly for the training and evaluation units (one Cholesky factor of K on their union; above 3,000 units a low-rank eigen approximation keeping 99.9% of the trace is used instead, and reported). Phenotypes are $y_S = \mu + g_S + e$, $e \sim N(0, \sigma_e^2 I)$ with $\sigma_e^2 = (1 - h^2)/h^2$ (the genetic variance is 1 on the package's kinship scale). Predictions are GBLUP with a fixed intercept,

$$\hat{g}_E = K_{ES} [V^{-1} - V^{-1}1(1'V^{-1}1)^{-1}1'V^{-1}] y_S, \quad V = K_{SS} + \sigma_e^2 I,$$

a weight matrix computed once per training set, so μ drops out.

Metrics per draw, with $k = \text{ceiling}(\text{top} * |E|)$:

- overlap: share of the predicted top k that is in the true top k (equal to the precision of the selection);
- ndcg: NDCG@k with gains $g - \min(g)$, as in `tso_evaluate()`;
- gain: realised gain, mean true value of the selected k minus the mean of E , in genetic standard deviations of E ;
- cor: correlation between predicted and true values in E .

The table `units` gives, for each evaluation unit, how often it was selected, how often it was truly in the top k , and the probability that it is truly top given that it was selected.

Random training sets are drawn from the same candidates (keeping any history). When the evaluation set is the complement of the training set (untargeted plans), each random set is scored on its own complement. They share the simulated genetic values with the plan (common random numbers), using `n_sim / 5` draws each.

Value

A `tso_topk` object: summary (mean and 90% interval of each metric, for the plan and for random sets), draws (per-draw metrics), `units` (per-unit selection probabilities), `k` and `spec`.

Examples

```
panel <- tso_simulate_panel(n = 150, markers = 400, seed = 1)
plan <- tso_design(panel, n = 30, verbose = FALSE)
tk <- tso_topk(plan, n_sim = 100, n_random = 5)
tk

plot(tk)
```

tso_trait_value

What is each trait worth? Proxy and high-throughput traits in multi-trait designs

Description

For a multi-trait plan from `tso_design()` (`tso_data(traits = )`), reports for every trait what the design spent on it, what one more measurement of it would add to the index reliability per unit of cost, and how much the index CDmean falls when the trait is dropped entirely and its budget is spent on the other traits. The **exchange rate** says how many measurements of a trait carry the information of one measurement of the reference trait (by default the most weighted trait in the index): the number of drone or NIRS scans that replace one yield plot.

Usage

```
tso_trait_value(plan, reference = NULL, redesign = TRUE)
```

Arguments

plan	A multi-trait tso_plan.
reference	The trait the exchange rate is expressed in (default: the trait with the largest index weight).
redesign	If TRUE (default), re-run the design without each trait (same budget or size, same targets) to measure what the trait is worth.

Details

Cheap, heritable traits genetically correlated with the index traits (NDVI, canopy temperature, NIRS predictions, early vigour) can be worth measuring even with index weight 0: they carry information about the traits that are selected on. This function measures how much.

Value

A tso_trait_value object: table (one row per trait), spec.

Examples

```
panel <- tso_simulate_panel(n = 150, markers = 500, seed = 1)
tt <- tso_data(panel, traits = c("yield", "ndvi"), trait_h2 = c(0.3, 0.7),
               gcor = matrix(c(1, 0.6, 0.6, 1), 2), trait_cost = c(20, 1),
               plot_cost = 5, verbose = FALSE)
plan <- tso_design(tt, budget = 1500, index = c(yield = 1, ndvi = 0), verbose = FALSE)
tso_trait_value(plan)
```

tso_transfer	<i>Across-programme transfer: can training in one programme predict another?</i>
--------------	--

Description

tso_transfer() answers a question every breeding network asks: if the training lines are phenotyped in programme A (from), how well are the lines of programme B (to) predicted, compared with training in B itself, and how many local "bridge" lines phenotyped in B recover the difference?

Usage

```
tso_transfer(data, from, to, n, n_local = 0, cost_local = 1, h2 = NULL)
```

Arguments

data	A tso_data object or anything <code>tso_data()</code> accepts.
from	Lines of programme A: IDs, a logical vector over units, or group name(s) matched against <code>data\$units\$group</code> .
to	Lines of programme B, in the same forms.
n	Number of training units (plots) in each design.
n_local	Maximum number of local bridge lines from B in the bridge design (default 0: no bridge).
cost_local	Cost of one plot in B relative to one plot in A (default 1).
h2	Assumed heritability; NULL averages over 0.2, 0.5 and 0.8.

Details

Three designs of the same size n are built with the CDmean engine of `tso_design()` (greedy robust CDmean with the exact fixed intercept; the targets are always the lines of B):

1. **from only**: the n lines are chosen from A.
2. **with bridge**: the lines are chosen from A and B together, with at most n_local of them from B. The cap is enforced inside the greedy search (an eligibility rule: once n_local lines of B are in the set, only lines of A remain eligible), so the search decides jointly which local lines to add and which lines of A they replace, rather than picking the bridge first and the rest afterwards. A plot in B costs $cost_local$ plots in A; the design spends a budget of n A-plots.
3. **in-programme reference**: the n lines are chosen from B itself, to predict the rest of B: the accuracy you would get without any transfer.

Each design is scored on the lines of B that it does not phenotype (a line is never scored as its own target): expected CDmean averaged over the heritabilities $h2$, and accuracy $\sqrt{CDmean}$. Transferability is $accuracy(from\ only) / accuracy(in-programme)$. The bridge curve repeats the bridge design for a small grid of caps from 0 to n_local .

Relatedness between the programmes is summarised on $R = cov2cor(K)$: for each line of B, its mean and maximum relationship to the lines of A (and, for comparison, its maximum relationship to the other lines of B), and the share of B lines with a relative ≥ 0.2 in A.

Value

A `tso_transfer` object: designs (data frame: `design`, `n_from`, `n_to`, `cd`, `accuracy`, `relative` = accuracy relative to the in-programme reference), `transferability`, `bridge_ratio` (accuracy of the bridge design relative to the in-programme reference), `bridge` (the bridge curve: `n_local`, `n_local_used`, `n_from`, `cd`, `accuracy`, `relative`), `relatedness` (list: lines data frame with `id`, `mean_to_from`, `max_to_from`, `max_within`; `mean`, `max`, `share_020`), `selected` (IDs chosen by each design), `scored` (IDs of B scored by each design) and `spec`.

Examples

```
panel <- tso_simulate_panel(n = 160, markers = 500, groups = 4, fst = 0.3, seed = 1)
tr <- tso_transfer(panel, from = c("G1", "G2", "G3"), to = "G4", n = 25, n_local = 8)
tr

plot(tr)
```

tso_uncertainty	<i>How sure is the plan's advantage? Marker-bootstrap uncertainty</i>
-----------------	---

Description

A plan from `tso_design()` reports one number for its advantage over random sampling, computed on one relationship matrix. That matrix is an estimate: built from a finite set of markers, it would change with another SNP chip. `tso_uncertainty()` asks whether the advantage survives that change. It bootstraps the markers, rebuilds the relationship matrix on each replicate, and recomputes both the plan's CDmean and that of random training sets of the same size, so the gain over random is re-evaluated on every replicate.

Usage

```
tso_uncertainty(plan, B = 50, n_random = 20, seed = 1, level = 0.9)
```

Arguments

plan	A tso_plan from <code>tso_design()</code> .
B	Number of marker-bootstrap replicates (default 50).
n_random	Random training sets per replicate (default 20).
seed	Random seed (the global random state is restored afterwards).
level	Coverage of the percentile intervals (default 0.9).

Details

Algorithm. For each of B replicates:

1. resample the marker columns of `plan$data$geno` (after its QC) with replacement, keeping the base allele frequency of each resampled marker;
2. rebuild the relationship matrix with `tso_kinship()` on those dosages and base frequencies (adding the dominance kernel with the same share when the data used one);
3. compute the exact CDmean of the plan on its targets: centred contrasts on `plan$index$target` (the reliability of the target when there is only one), residual variance from the plan's heritability (`plan$spec$h2`, the median of the grid for robust designs), the intercept projected out (unless the plan used `fixed = "none"`), and historical records weighted as in the plan;
4. draw `n_random` random training sets of the same size from the plan's candidates (excluding its must-include and historical units, which are kept in every set) and average their CDmean;
5. record the gain in accuracy, on the square-root scale used by `tso_design()`: $\sqrt{\text{cd_plan}} / \text{mean}(\sqrt{\text{cd_random}}) - 1$.

Percentile intervals at `level` are reported for the plan's CDmean, the random mean, and the gain; with the probability that the plan beats the random mean. Resampling markers with replacement makes every bootstrap relationship matrix a little noisier than the original, which shifts all CDmeans by a similar amount; the intervals are therefore the percentile intervals of the replicates re-centred on the estimate from all markers, and the shift is reported in the `bias` column. `draws` keeps the raw replicate values. **Verdict:** "robust" when the lower bound of the accuracy gain is above 0.02; "uncertain" when the interval contains 0; "not supported" when the upper bound is below 0; "small" when the whole interval is positive but its lower bound is at most 0.02.

Relation to `tso_design()`. The plan's own verdict answers "how much better than random, assuming this relationship matrix is right?"; this function answers "would the answer change with other markers?". The set is not re-optimised on each replicate: it is the plan you would plant, scored under relationship matrices it was not chosen for. Replicate CDmeans are therefore slightly lower than the plan's reported value (the optimiser's winner's curse), which is part of what is being measured.

Supported plans. Plans whose units are genotyped lines: systems "inbred", "clonal", "outcross" and "genebank", without environments or traits, built from marker genotypes. Hybrid, multi-environment, multi-trait and kinship- or pedigree-only plans give an error. Major-locus weighting and user-supplied kinships are not rebuilt: the bootstrap uses the genome-wide marker kernel (a warning says so). Extra fixed effects beyond the intercept are not re-applied.

Value

A `tso_uncertainty` object: draws (one row per replicate), intervals (estimate on all markers, lower and upper bounds), `p_beats` (share of replicates where the plan beats the random mean), verdict (label and text), original (the plan's reported values) and spec. Print it, or plot it with `plot()`.

Examples

```
panel <- tso_simulate_panel(n = 150, markers = 500, seed = 1)
plan <- tso_design(panel, n = 30, verbose = FALSE)
u <- tso_uncertainty(plan, B = 20, n_random = 10)
u
plot(u)
```

tso_update

Refresh the training set for a new season

Description

Carries a previous plan forward: the units phenotyped before become historical data (informative, free, down-weighted by `history_weight`), and a new set is chosen among the new candidates. Base-population allele frequencies from the previous data are reused automatically, so relationships stay comparable across cycles.

Usage

```
tso_update(previous, geno = NULL, data = NULL, n, history_weight = 0.8, ...)
```

Arguments

<code>previous</code>	A <code>tso_plan</code> from an earlier season.
<code>geno</code>	Genotypes of the whole current panel (old and new lines), in any format <code>tso_read()</code> accepts. Alternatively give data.
<code>data</code>	A <code>tso_data</code> object for the current panel.
<code>n</code>	Number of new units to phenotype.
<code>history_weight</code>	Weight of last season's records (default 0.8).
<code>...</code>	Other arguments passed to <code>tso_design()</code> .

Value

A new tso_plan.

Examples

```
panel <- tso_simulate_panel(n = 200, markers = 500, seed = 1)
old <- tso_design(panel, n = 30, candidates = 1:100, verbose = FALSE)
new <- tso_update(old, geno = panel, n = 20)
```

tso_variance	<i>Estimate variance components from historical data</i>
--------------	--

Description

Fits the genetic model that [tso_data\(\)](#) designs under, by restricted maximum likelihood (REML) with the average-information algorithm, on your historical trials:

- several **traits** (traits = two or more columns; one row per line): genetic and residual covariance matrices between traits, hence genetic correlations and heritabilities;
- several **environments** (env = a column; one row per line and environment, entry means): the genetic covariance between environments and a residual variance per environment;
- several **kernels** (kernels =): the share of genetic variance of each.

Usage

```
tso_variance(
  geno,
  pheno,
  traits = "y",
  env = NULL,
  kernels = NULL,
  draws = 5,
  maxit = 50,
  tol = 1e-06,
  verbose = FALSE
)
```

Arguments

geno	Genotypes (tso_geno, a file) or a tso_data, whose additive relationship matrix is used (unless kernels).
pheno	Data frame with an id column (line IDs) and the trait column(s); with env, one row per line and environment.
traits	Trait column(s). Default "y".
env	For multi-environment data: the environment column.
kernels	For kernel shares (one trait): kernel names, a tso_kernels() object or a named list of matrices.
draws	Scenarios drawn from the sampling distribution (default 5).
maxit, tol	Iteration limit and convergence tolerance on the log-likelihood.
verbose	Print iterations.

Details

Standard errors come from the inverse of the average-information matrix, and draws scenarios are sampled from that sampling distribution. `tso_data(variance = )` then uses the estimates and the robust designs average over the drawn scenarios instead of fixed grids.

Value

A `tso_variance` object: `type`, `groups`, `G` (genetic covariance), `R` (residual covariance), `gcor` (genetic correlations), `h2` (heritability per group), `share` (kernels), `se`, `loglik`, `converged`, `iterations` and `scenarios` (the draws).

Examples

```
hist <- tso_simulate_cycles(cycles = 2, n_candidates = 120, markers = 400, seed = 1)
v <- tso_variance(hist$geno, hist$pheno)
v
```

tso_verify

Check that a plan was made from these data

Description

Recomputes the fingerprints of the data (and, with `redesign = TRUE`, re-runs the design with the plan's settings) and reports what matches.

Usage

```
tso_verify(plan, data)
```

Arguments

<code>plan</code>	A <code>tso_plan</code> (or a manifest from <code>tso_manifest()</code> / a plan file from <code>tso_plan_json()</code>).
<code>data</code>	The <code>tso_data</code> the plan should come from.

Value

A data frame: `item`, `recorded`, `current`, `match`.

tso_write*Write genotypes to another format*

Description

Saves a `tso_geno` object (or a dosage matrix) as a table, VCF, HapMap, PLINK binary or R file. The format follows the file extension; add `.gz` to compress text formats.

Usage

```
tso_write(x, file, format = "auto")
```

Arguments

<code>x</code>	A <code>tso_geno</code> object, or a dosage matrix (lines x markers).
<code>file</code>	Output file name.
<code>format</code>	"auto" (from the extension) or an explicit format name.

Details

Extension	Format written
<code>.csv, .tsv, .txt</code>	lines x markers table of dosages, first column id
<code>.vcf</code>	VCF 4.2 with GT (diploid and polyploid)
<code>.hmp.txt</code>	HapMap with two-letter calls (diploid)
<code>.bed</code>	PLINK binary set (<code>.bed</code> , <code>.bim</code> , <code>.fam</code> ; diploid)
<code>.raw</code>	PLINK additive format
<code>.rds</code>	the <code>tso_geno</code> object itself

Value

The file name(s) written, invisibly.

Examples

```
g <- tso_simulate_panel(n = 20, markers = 50)
f <- tso_write(g, tempfile(fileext = ".vcf.gz"))
tso_read(f)
```

Index

`plot.tso_pareto(tso_pareto)`, 55
`print.tso_pareto(tso_pareto)`, 55

`tso_analysis_script`, 4
`tso_backtest`, 4
`tso_backtest()`, 70
`tso_brapi_connect`, 6
`tso_brapi_connect()`, 7, 9–12
`tso_brapi_geno`, 7
`tso_brapi_geno()`, 6
`tso_brapi_pheno`, 8
`tso_brapi_pheno()`, 6, 7, 10
`tso_brapi_studies`, 9
`tso_brapi_studies()`, 6, 9
`tso_brapi_trait`, 10
`tso_brapi_trait()`, 6, 9, 12
`tso_brapi_variantsets`, 11
`tso_brapi_variantsets()`, 6, 8
`tso_brapi_write_list`, 11
`tso_brapi_write_units`, 12
`tso_brapi_write_units()`, 12
`tso_cli`, 13
`tso_colors`, 13
`tso_compare`, 14
`tso_compare()`, 60
`tso_contribution`, 15
`tso_contribution()`, 60
`tso_criteria`, 17
`tso_criteria()`, 5
`tso_crosses`, 17
`tso_data`, 19
`tso_data()`, 7, 16, 24–26, 29, 31, 35, 37, 40, 41, 46–48, 52–54, 56, 57, 62, 64, 67, 75, 78, 81
`tso_design`, 23
`tso_design()`, 5, 7, 14, 17, 20–22, 27, 34, 37, 38, 41, 43, 44, 51, 54, 55, 57, 64, 67, 75, 76, 78–80
`tso_design_large`, 27
`tso_design_large()`, 60
`tso_diagnose`, 28
`tso_diagnose()`, 60
`tso_docs`, 30
`tso_economics`, 30
`tso_evaluate`, 32
`tso_evaluate()`, 10, 76
`tso_explain`, 33
`tso_export`, 33
`tso_export()`, 26, 34
`tso_export_field`, 34
`tso_genotype_set`, 35
`tso_genotype_set()`, 38, 54
`tso_haplotypes`, 37
`tso_haplotypes()`, 21, 40, 41, 57
`tso_imputation_plan`, 38
`tso_imputation_plan()`, 54
`tso_kernels`, 40
`tso_kernels()`, 21, 37, 81
`tso_kinship`, 42
`tso_kinship()`, 7, 16, 28, 79
`tso_layout`, 43
`tso_layout()`, 34
`tso_licence`, 45
`tso_licence()`, 46
`tso_licence_request`, 46
`tso_licence_request()`, 45
`tso_major_effects`, 46
`tso_major_effects()`, 18, 21, 22, 49, 60
`tso_major_scan`, 48
`tso_major_scan()`, 10, 21, 22, 42, 46–48, 60
`tso_manifest`, 50
`tso_manifest()`, 59, 82
`tso_next_batch`, 50
`tso_next_batch()`, 60
`tso_omics_kernel`, 52
`tso_omics_kernel()`, 21, 55, 60
`tso_omics_plan`, 54
`tso_omics_plan()`, 54, 60
`tso_pareto`, 55
`tso_pareto()`, 58, 60
`tso_pareto_pick`, 58
`tso_pareto_pick()`, 55, 57
`tso_plan_json`, 59
`tso_plan_json()`, 82
`tso_plot`, 59
`tso_plot()`, 26, 29, 70
`tso_qc`, 60

tso_qc(), [20](#), [27](#), [38](#), [42](#), [62](#)
tso_read, [61](#)
tso_read(), [8](#), [20](#), [27](#), [37](#), [38](#), [47](#), [49](#), [61](#), [80](#)
tso_report, [63](#)
tso_sample_size, [64](#)
tso_sample_size(), [30](#), [31](#), [60](#)
tso_save_figure, [65](#)
tso_save_figure(), [59](#)
tso_score, [66](#)
tso_season, [67](#)
tso_simulate, [69](#)
tso_simulate(), [60](#)
tso_simulate_cycles, [70](#)
tso_simulate_omics, [71](#)
tso_simulate_panel, [72](#)
tso_simulate_panel(), [72](#)
tso_simulate_trait, [73](#)
tso_theme, [74](#)
tso_theme(), [56](#), [59](#)
tso_topk, [75](#)
tso_topk(), [60](#)
tso_trait_value, [76](#)
tso_transfer, [77](#)
tso_uncertainty, [79](#)
tso_update, [80](#)
tso_variance, [81](#)
tso_variance(), [21](#), [54](#), [55](#)
tso_verify, [82](#)
tso_verify(), [50](#)
tso_write, [83](#)
tso_write(), [62](#)